KOVA

A Complete Manual

The language, the architecture, and the ideas behind it

Samuel Ifedapo Oguntimehin

github.com//kova-lang

Version 0.5.0 | March 2026

Made out of curiosity.

Table of Contents

Section 1 ..... **8**
What is Kova? **8**
1.1 The One-Sentence Answer 8
1.3 The Problem Kova Was Built to Solve 9
1.4 What Kind of Language is Kova? 11
1.4.1 A Domain-Specific Language (DSL) 11
1.4.2 An Interpreted Language 11
1.4.3 A Statically Typed Language (for AI values) 11
1.4.4 A Research-Oriented Language 11
1.5 What Kova Looks Like 12
1.6 The Execution Graph 13
1.7 What Kova is Not 14
1.8 Who Built Kova and Why 15
1.9 The Current State of Kova 15
Why Kova Exists: The Problem It Solves **17**
2.1 How Software Used to Work 17
2.2 What Changed: AI Models as Program Components 18
2.3 The Invisible Uncertainty Problem 18
2.4 Why Existing Solutions Are Not Enough 19
2.4.1 Comments and Documentation 20
2.4.2 Naming Conventions 20
2.4.3 Wrapper Types in TypeScript 20
2.4.4 Logging and Observability Tools 20
2.5 The Deeper Problem: Auditability 20
2.6 The Research Question Kova Answers 21
2.7 The Control Systems Parallel 22
2.8 What Kova Does Not Claim to Solve 23
2.9 Summary: The Problem and the Solution 23
Why Kova is Different from Other Languages **24**
3.1 The Central Difference 25
3.2 Kova vs General-Purpose Languages 25
3.2.1 Python 25
3.2.2 JavaScript / TypeScript 26
3.2.3 Rust 27
3.3 Kova vs AI Orchestration Frameworks 28

3.3.1 LangChain....28
3.3.2 LlamaIndex and Similar Tools....29
3.4 Kova vs Other DSLs....29
3.4.1 SQL....29
3.4.2 GraphQL....29
3.4.3 Jsonnet / Dhall....29
3.5 The Comparison Table....30
3.6 What Kova Gives Up....31
3.7 The Broader Point: Languages Shape Thinking....31
Related Research....32
4.1 Probabilistic Programming Languages....33
4.1.1 Church and Anglican....33
4.1.2 Stan....34
4.1.3 Pyro and NumPyro....34
4.2 Type Systems and Effect Systems....34
4.2.1 Dependent Types....34
4.2.2 Effect Systems....35
4.2.3 Information Flow Types....35
4.3 Neuro-Symbolic Computing....35
4.3.1 The Neuro-Symbolic Spectrum....36
4.3.2 Symbolic Scaffolding for Neural Models....36
4.4 AI Safety and Formal Verification....37
4.4.1 Certified Machine Learning....37
4.4.2 Responsible AI and Auditability....37
4.5 Program Analysis and Execution Graphs....37
4.5.1 Control Flow Graphs....38
4.5.2 Data Flow Graphs....38
4.5.3 Computation Graphs in Deep Learning....38
4.6 Reinforcement Learning and Decision Making Under Uncertainty....39
4.6.1 Markov Decision Processes....39
4.6.2 Partially Observable MDPs....39
4.7 Domain-Specific Languages for AI Workflows....39
4.7.1 Halide....39
4.7.2 ONNX and ML Interchange Formats....40
4.8 Where Kova Stands in This Landscape....40
Prerequisites....41
5.1 To Use Kova: Minimum Requirements....42
5.1.1 Basic Programming Concepts....42

5.1.2 What an API Is....43
5.1.3 What an AI Model Is (Loosely)....44
5.1.4 Node.js Installed....44
5.2 To Understand the Kova Pipeline: Intermediate Requirements....44
5.2.1 What a Compiler and Interpreter Are....45
5.2.2 What Tokens Are....45
5.2.3 What an Abstract Syntax Tree Is....46
5.2.4 What Semantic Analysis Is....47
5.2.5 What Interpretation Is....47
5.2.6 What a Type System Is....48
5.2.7 What a Graph Is (in Computer Science)....48
5.3 To Build or Extend Kova: Advanced Requirements....49
5.3.1 JavaScript and Node.js (Intermediate Level)....49
5.3.2 Formal Grammars....49
5.3.3 Tree Data Structures....50
5.3.4 Scope and Symbol Tables....50
5.3.5 Graph Algorithms....51
5.4 Recommended Learning Resources....51
5.4.1 For Programming Language Implementation....51
5.4.2 For Type Systems....52
5.4.3 For Graphs and Algorithms....52
5.4.4 For JavaScript....52
5.5 What You Do Not Need....52
How Kova Works: The Full Pipeline....54
6.1 The Lexer....54
6.1.1 What It Does....54
6.1.2 How the Kova Lexer Works....55
6.1.3 Skipping Ignorable Characters....57
6.1.4 Reading Different Token Types....57
6.1.5 What the Lexer Produces....58
6.2 The Parser....58
6.2.1 What It Does....58
6.2.2 The Entry Point....58
6.2.3 Expression Parsing and Precedence....60
6.2.4 Error Handling in the Parser....61
6.3 The Abstract Syntax Tree....61
6.3.1 What It Is AST?....61
6.3.2 AST Node Structure....62

6.3.3 Why the AST Matters ... 63
6.4 The Semantic Analyzer ... 64
6.4.1 What It Does ... 64
6.4.2 Scope Implementation ... 64
6.4.3 The Prob<T> Type System ... 65
6.4.4 Error Reporting ... 67
6.5 The Interpreter ... 67
6.5.1 What It Does ... 68
6.5.2 Interpreter State ... 68
6.5.3 The Visit Method ... 69
6.5.4 Built-in Functions ... 71
6.5.5 HTTP, Database, and Respond Handling ... 71
6.5.6 Function Calls and Closures ... 72
6.6 The Execution Graph ... 72
6.6.1 What It Does ... 72
6.6.2 Node Kinds ... 73
6.6.3 Edge Kinds ... 74
6.6.4 How the Graph Is Built ... 74
6.6.5 Topological Ordering ... 75
6.6.6 Parallel Candidates ... 75
6.6.7 Graph Output Format ... 76
6.7 The AI Layer and Prob<T> ... 77
6.7.1 Overview ... 77
6.7.2 The Prob<T> Object ... 77
6.7.3 makeProb and stubAI ... 78
6.7.4 resolveProb ... 79
6.7.5 groqAI: The Live AI Call ... 80
6.7.6 Structured Output ... 81
6.7.7 Auto-Detection of Mode ... 82
6.8 The Full Pipeline: End to End ... 83
The CLI ... 85
7.1 Installation ... 85
7.1.1 Requirements ... 85
7.1.2 Installing from Source ... 85
7.1.3 Verifying the Installation ... 86
7.1.4 nvm PATH Issue ... 87
7.2 Commands ... 87
7.3 kova run ... 88

7.3.1 Basic Usage....88
7.3.2 Stub Mode....89
7.3.3 File Path Resolution....89
7.4 kova run --graph....90
7.4.1 Reading the Graph Output....91
7.5 kova check....92
7.6 Setting Up the Groq API Key....93
7.6.1 Getting a Key....93
7.6.2 Adding the Key....94
7.6.3 Keeping the Key Safe....94
7.7 Error Messages....94
7.7.1 Syntax Errors....95
7.7.2 Semantic Errors....95
8.7.3 Runtime Errors....95
7.8 Using Kova Programmatically....96
7.9 Running the Test Suite....97
7.10 Project Structure Reference....98
Limitations....101
8.1 Language Scope....101
8.2 Runtime and Performance....102
8.3 The Module System....103
8.4 HTTP and Database Stubs....103
8.5 Type System Completeness....104
8.6 Error Handling....105
8.7 The Parallel Execution Gap....105
8.8 No Package Ecosystem....106
8.9 Single AI Provider....106
8.10 The Prob<T> Confidence Field....107
8.11 What Kova Is Not Trying to Fix....107
What Kova Could Be....109
9.1 Near-Term: Completing the Implementation....109
9.2 Medium-Term: Architectural Extensions....112
9.3 Research Directions....113
9.4 The Longer Vision....115
Appendix and Glossary....116
A.1 Complete Keyword Reference....116
A.2 Complete Operator Reference....118
A.3 Error Message Reference....119

Semantic Errors....119
Runtime Errors....120
A.4 Quick-Start Cheat Sheet....121
A.5 Glossary....123
A.6 Bibliography....127

What is Kova?

1.1 The One-Sentence Answer

Kova is a programming language I built from scratch and what it does is, it treats AI uncertainty as a structural property of programs, and not an afterthought.

Yes I am aware, there are a lot to the sentence, and please do not worry, I will be de-constructing it for you and by the time that we have reached the end of this section, you shall know: What Kova is, what problem Kova solves and the reason why I built it.

1.2 Let's start from the beginning: What is a Programming Language?

Of course, before telling you what Kova is, we need to get to agree on what a programming language is.

Fundamentally, programming languages are a set of orders that are understood by the computer. Writing a program is telling the computer what it has to do, step by step. This language has its rules, like grammar, we refer to this as syntax and this syntax has a meaning known as semantics. If you break the rules, then the computer will not understand your instructions. Follow them and the computer executes your instructions as intended.

Languages like Python, JavaScript, and Java are general-purpose languages. They are good for almost any computation. Kova is different, it is built for backend systems that use AI models. It is narrow by design for a reason, that is the reason. It is a domain specific language, in a word.

> **Analogy:** *A hammer and file are both hand tools. The file is narrow and the narrowness is not a limitation but rather precise enough for file-ing. Kova is a file for AI embedded backend software.*

1.3 The Problem Kova Was Built to Solve

Let us understand why Kova exists. To do that we need to recognize a problem that appears when you write a code that calls AI models.

When a program calls an AI model for example classifying a text data as either positive or negative the result comes back as a value. The thing with this value is that it is like any other value in most programming languages. It could be string, number or an object. The issue with that is that the language doesn't realize the number came from an AI. Or even worse, that the number is probabilistic. It treats the number just like a database query, or any other mathematical calculation that would have produced it.

This creates a problem. AI outputs are not certain. They are probabilistic. The model made its best guess. It could be wrong. But the program does not track this. The uncertainty is invisible to the language.

> **The problem in plain English:** *When your program asks an AI a question and uses the answer to make a decision, the language you wrote it in has no way of knowing that the answer might be wrong. It treats AI output like fact.*

Here is what this looks like in a typical language:

```
// JavaScript:  typical AI integration

const result = await ai.classify(text);

if (result === 'positive') {

        approveComment();   // what if the AI was wrong?

}
```

There is nothing in this code to show that the result might be wrong. The programmer knows it. The programming language does not. The part of the language that checks what kind of value something is, just sees a string. It has no way to say that this string might not be right. So you have to be careful.

Kova was made to solve this problem. In Kova every output from AI has its type. It's not a string. It's not a number. It's a type called Prob<T>, which is short for a value that has some uncertainty. It is not like a value. You cannot put it inside an if-statement. You cannot pass it to a function which accepts a value. It cannot be used until it is called with resolve. Calling it is like stating: I know the value is an AI one and that it can be incorrect. I'm okay with it and will work with it.

```
// Kova:  AI uncertainty is structural

let probSentiment = AI("classify sentiment", text)

// probSentiment is Prob<T>:  cannot use directly


let sentiment = resolve(probSentiment)

// sentiment is now a string i.e uncertainty acknowledged


if sentiment == "positive" {

        respond { status: 200, body: { approved: true } }

}
```

The language enforces this. If you try to use a Prob<T> value in a condition without resolving it, the semantic analyzer rejects your program before it even runs. You are forced to be explicit.

1.4 What Kind of Language is Kova?

Kova is several things at once. To understand it as a whole, we need to understand each aspect of it.

1.4.1 A Domain-Specific Language (DSL)

A **domain-specific language** is language used in a particular problem domain rather than as a general purpose computation language. For example, SQL is a DSL for querying databases. HTML is a DSL for describing web page structure. Kova is a DSL for AI-integrated backend workflows. In it, it has syntax for HTTP requests, database operations, and AI calls because those are the operations its domain requires.

1.4.2 An Interpreted Language

Kova is an **interpreted language**. This means that it does not compile to machine code or another language before execution. It runs directly through its own interpreter. It's straightforward, the program reads Kova code and executes it. This is the kind of model used by Python and Ruby. Kova's interpreter is written in JavaScript and runs on Node.js.

1.4.3 A Statically Typed Language (for AI values)

Most of Kova is dynamically typed, and what this means is that you do not have to declare what type of value a variable is. But AI values are **statically checked**. In the semantic analyzer (which happens in the pipeline before runtime, the Prob<T> values are checked never to be used while unbound, or in an unresolved way): This combines freedom for generic code, with constraints where necessary.

1.4.4 A Research-Oriented Language

Kova is not primarily or yet a production tool. It is a proof of concept for a research question: can a programming language's type system make AI uncertainty visible, auditable, and structurally enforced? The answer, demonstrated by Kova, is yes.

1.5 What Kova Looks Like

Here is a complete Kova program. Read it as plain English first. I will break down every piece in later sections.

```
// Support ticket routing pipeline


// Variable declaration

let ticket = "I have been charged twice and want a refund immediately."


let probUrgency = AI("classify urgency as: low, medium, high, or critical", ticket)

let probDepartment = AI("classify department as: billing or technical", ticket)

let probSentiment  = AI("classify sentiment as: calm, frustrated, or angry", ticket)


let urgency = resolve(probUrgency)

let department = resolve(probDepartment)

let sentiment  = resolve(probSentiment)


if urgency == "critical" {
```

```
        POST "/alerts" save { message: ticket, urgency: urgency }
}


respond {
        status: 200,
        body: {
        queue:   department + "-" + urgency,
        escalate:   sentiment == "angry",
        department: department
        }
}
```

We need to notice that some things were not included in this program: import statements for HTTP libraries, try-catch blocks for API errors, manual type casting, or boilerplate setup. HTTP, AI, and respond are built into the language. The uncertainty of the AI calls is visible in the type of each variable. The structure of the program maps directly onto the logic of the task.

1.6 The Execution Graph

Beyond the result, every kova program produces an execution graph. A graph is made up of edges and nodes. This is a directed graph that represents the control flow and data flow of the program.

Each operation in the program becomes a node. Assignments become declare nodes. AI calls become ai nodes. resolve() calls become resolve_prob nodes. HTTP requests become http nodes. The edges between the nodes show how data flows and what depends on what.

The graph serves several purposes. It makes the program's structure visible and auditable. It can be analysed to find operations that could run in parallel, and it maps directly onto structures studied in formal AI research: policy graphs, computation graphs, and probabilistic graphical models.

This is not a coincidence. It is the central research contribution of Kova: a symbolic program that calls AI models naturally produces a graph structure that encodes the relationship between deterministic logic and probabilistic inference. The language makes this structure visible by design.

> **In plain terms:** *When you run a Kova program, you do not just get an output. You get a map of every decision the program made and every place uncertainty entered it. That map is the execution graph.*

1.7 What Kova is Not

It is as important to say what Kova is not as what it is.

- Kova is not a general-purpose language. You would not write a game, a desktop application, or a data science pipeline in Kova. It is designed for one narrow domain.
- Kova is not a production-ready framework. It is a proof of concept and a research tool. It has not been hardened for enterprise use.
- Kova is not a wrapper around another language. It is not JavaScript with AI helpers. It is a separate language with its own lexer, parser, semantic analyzer, interpreter, and type system, built from first principles.
- Kova is not trying to replace existing AI orchestration tools. It is trying to ask a deeper question: what would it look like if the language itself enforced good practices around AI uncertainty, rather than leaving it to the programmer?

1.8 Who Built Kova and Why

Kova was developed by Samuel Ifedapo Oguntimehin, who studied Mechanical Engineering at Obafemi Awolowo University, Nigeria. This product was created without supervision or a formal body and it began development in 2025.

The motivation came from two directions colliding. He spent many years constructing backend systems and he ran into a real problem: "If a piece of code calls an AI model and the returned data is used for a decision making, then what assurance does he have that this decision will be good?" in almost any language the answer is none. The language offers no structural help.

The Mechanical Engineering background added a second frame. Control systems deal with uncertainty constantly. State estimation, feedback loops, sensor noise, these are solved problems with formal mathematical structures. The observation that an AI model estimating an output and a Kalman filter estimating a state are structurally similar problems led to the central question Kova explores: can the type system of a programming language do what a control system does for physical systems i.e make uncertainty visible, bounded, and formally tracked?

Kova is the answer to that question in code.

1.9 The Current State of Kova

As of version 0.5.0: Complete, functioning language, includes:

- A full interpreter pipeline: lexer, recursive-descent parser, semantic analyzer, async interpreter
- A first-class probabilistic type system with Prob<T> and resolve()
- Live AI integration via Groq (llama-3.3-70b-versatile), with stub mode for offline development
- Built-in HTTP verbs: GET, POST, PUT, DELETE, PATCH
- Built-in database operations: connect, find, insert, update

- Environment variable support via ENV.KEY
- Module imports
- An execution graph engine that produces structured node/edge graphs
- A CLI with run, check, and --graph commands
- 117 tests passing, covering the full language feature set

The language is open source at github.com/kova-lang. The manual you are reading explains every part of it in depth.

Next: Section 2 - Why Kova Exists

Section 2

Why Kova Exists: The Problem It Solves

Every programming language exists because someone noticed a gap. C existed because programmers needed control over hardware without writing assembly. Python existed because programmers needed expressiveness without the weight of C. Kova exists because of a specific gap that opened up when AI models became part of everyday software.

This section explains that gap in full. We start from the basics and build up to the exact problem Kova addresses.

2.1 How Software Used to Work

For quite some time programs have been deterministic. Deterministic implies that when the same input into a program you will get the same output every single time. This means that when using an adder program you will always receive the correct output, similarly when using a program to sort a lot of numbers together you will also always get the correct sorted output. There was never any room for uncertainty, randomness, or guessing in a program.

This was also a benefit to understanding the program. You could understand the program based on its code. This was also useful for developing a set of tests that would check for a specific output. It was also useful for developing a set of type systems that would check for the structural integrity of the program before it was executed.

> **Key idea:** *Deterministic programs are predictable. The same input always produces the same output. This predictability is what makes them easy to test, verify, and trust.*

2.2 What Changed: AI Models as Program Components

AI models became fast enough, cheap enough, and advanced enough that programmers began to start using them as a component within a normal program. Not as a standalone product, but as a component that a program can call in the middle of execution.

A program can call one to classify a support ticket. Another program can call one to read some data out of a document. Yet another program can call one to make a determination about whether some content is safe to publish.. These calls happen inside programs that also read databases, call APIs, check conditions, and respond to users.

This has created a new kind of program. It is partly deterministic and partly probabilistic. The deterministic portion of the program will always behave in the same way. The probabilistic portion will not. It will make its best guess. It can be wrong. It can be inconsistent. It can produce different outputs for the same input.

> **The new reality:** *Modern software is no longer purely deterministic. When a program calls an AI model, it introduces uncertainty into its own execution. The program is making decisions based on guesses.*

2.3 The Invisible Uncertainty Problem

This is where things start to go wrong.

When an AI model returns a result from a program, that result is just like any other result. It is a string, it is a number, or an object. The language does not provide any way to treat it as something different from a result that was mathematically derived or read directly from a database that is known to be correct or better say, certain. The language does not provide any way to indicate: this result is uncertain, it might be wrong, treat it carefully.

What this means, basically, is that uncertainty is transparent in the language. The uncertainty is in reality, but it is not in the language; the programmer is aware that the AI could be wrong but

there is nothing in the language that can remind them that it could be wrong, or forces them to consider it, or keeps track of it. This leads to a class of bugs which are difficult to detect, and easy to create. Let's consider an example:

```
// A content moderation system in JavaScript
const label = await ai.classify(userComment);

if (label === 'safe') {
        publishComment(userComment);  // published based on AI guess
} else {
        blockComment(userComment);
}
```

The label variable is an AI result. The program uses it to make an important decision: publish or block. But nothing in the language indicates that it is an uncertain string. Nothing in the language indicates that it came from an AI. Nothing in the language requires the programmer to consider what to do if the AI was wrong.

The programmer might handle errors consciously. Or they might forget. Or a different programmer might modify the code later without knowing the value came from AI. The language offers no structural guarantee either way.

2.4 Why Existing Solutions Are Not Enough

This is not a new observation. Developers have tried to address it in several ways. None of them solve the structural problem.

2.4.1 Comments and Documentation

Any programmer can write a comment: // **this value came from AI, handle with care.** But then another programmer alters the code, or the comment and the code fall out of sync. Comments cannot be enforced. The language ignores them.

2.4.2 Naming Conventions

Some teams name AI-derived variables with a prefix, like ai_label or predicted_sentiment. This is better than nothing, but it is also just a convention. The language does not know what the prefix means. It does not prevent you from using the value without any special handling.

2.4.3 Wrapper Types in TypeScript

A TypeScript programmer might define a type like type AIResult<T> = { value: T; confidence: number } and use it to wrap AI outputs. This is the closest existing approach to what Kova does. But it is still optional. The type system does not prevent you from unwrapping it carelessly. There is no resolve() equivalent that is enforced at the language level. And adding this pattern to an existing codebase requires discipline across the whole team.

2.4.4 Logging and Observability Tools

Tools like LangSmith and other AI observability platforms let you trace AI calls after the fact. They show you what the model returned and when. But they operate at the monitoring level, not the language level. They tell you what happened. They do not prevent the structural problem from occurring in the first place.

> **The gap:** *All existing approaches address the symptom by asking programmers to be careful. None address the cause by making the language itself aware of uncertainty.*

2.5 The Deeper Problem: Auditability

Beyond the immediate risk of using an uncertain value carelessly, there is a longer-term problem: auditability.

As AI systems make more consequential decisions, there is increasing pressure to be able to explain those decisions. This is where it's essential, in the fields of regulated industries like finance, healthcare, law, etc. Where you need to be able to provide proof that you made what decision, on what data and if that data was factual.

With current tools, tracing the influence of an AI call through a program is difficult. The value enters the program, gets passed around, influences conditions and outputs, and by the time you are trying to audit a decision, the trail is hard to follow. The language never tracked it.

Kova addresses this through its execution graph. Because the language tracks every AI call as a distinct node type, and tracks every resolve() as another distinct node type, the graph of any Kova program shows exactly where uncertainty entered, how it propagated, and where it was resolved. The audit trail is structural, not bolted on after the fact.

2.6 The Research Question Kova Answers

Kova was built to answer a specific research question:

> *" Can a programming language's type system make AI uncertainty structurally visible, enforced, and auditable, in the same way that type systems make other program properties visible and enforced? "*

The answer Kova demonstrates is yes. Here is how it works in practice:

- Every call to AI() returns a value of type Prob<T>, not a plain string or number. The type encodes the fact that this value is uncertain.
- The semantic analyzer, which is executed prior to running your code, ensures that a Prob<T> value is not being placed in a condition, expression or used in a function call that takes a non-Prob type. When you do, it will fail the compilation.

- The only way you can get a non-Prob type value out of a Prob<T> is by calling resolve(). This is an explicit acknowledgement in the code that: I realize that this was generated by AI. I am accepting the uncertainty and proceeding.
- The execution graph records every AI call as an ai node and every resolve() as a resolve_prob node. The structure of uncertainty in the program is captured in the graph and can be inspected, visualised, and reasoned about.

This is not a workaround or a convention. It is enforced by the language. A Kova program that mishandles AI uncertainty cannot be compiled. The language makes the correct behaviour the only possible behaviour.

2.7 The Control Systems Parallel

There is a useful analogy from engineering that illuminates what Kova is doing.

In control systems, a system receives sensor measurements and uses them to make decisions. The problem is that sensor measurements are never perfectly accurate. They contain noise. A naive system that treats sensor data as ground truth will make poor decisions whenever the sensor is off.

The solution, developed over decades of control engineering, is to model the uncertainty explicitly. Techniques like Kalman filtering maintain a probability distribution over the estimated state, rather than a single point estimate. The uncertainty is tracked, not ignored. Every decision the system makes is informed by how confident it is in its current state estimate.

Kova applies the same logic to programs. An AI model is, in a precise sense, a sensor. It takes input and produces an estimate of something: sentiment, category, risk level, summary. That estimate is probabilistic. A program that treats it as ground truth is making the same mistake as a control system that ignores sensor noise.

Prob<T> and resolve() are the programming language equivalent of explicit uncertainty tracking in control systems. The language forces you to acknowledge the probabilistic nature of AI output before acting on it, just as a good control system maintains and propagates its uncertainty estimates rather than collapsing them prematurely.

> **In plain terms:** *A Kova program treats AI outputs the way a good engineer treats sensor readings: as estimates that carry uncertainty, not facts that can be used directly.*

2.8 What Kova Does Not Claim to Solve

Kova solves the structural visibility problem. It does not solve everything.

- It does not make AI models more accurate. The model can still be wrong. Kova just ensures that the program acknowledges this rather than ignoring it.
- It does not automatically tell you what to do when an AI is wrong. That is still a design decision for the programmer. Kova gives you the structure to make that decision explicitly.
- It does not solve the problem of AI systems that are biased, manipulated, or producing harmful outputs. Those are deeper problems that operate at the model level, not the language level.
- It does not replace the need for testing and validation of AI-integrated systems. It complements them by making the structure of uncertainty visible.

2.9 Summary: The Problem and the Solution

Here is the core problem and solution in the simplest possible terms:

The Problem

Modern software calls AI models. AI models produce uncertain outputs. Programming languages treat those outputs as certain. This makes uncertainty invisible at the structural level, which leads to programs that use AI outputs carelessly and audit trails that are hard to reconstruct.

The Solution

Kova introduces a first-class probabilistic type, Prob<T>, that every AI call returns. The type system enforces that this type cannot be used without explicit resolution. The execution graph records every point where uncertainty enters and leaves a program. The result is a language in which AI uncertainty is structurally visible, enforced, and auditable by design.

Next: Section 3 - Why Kova is Different from Other Languages

Section 3

Why Kova is Different from Other Languages

There are thousands of programming languages. A new one needs a reason to exist. This section explains precisely what makes Kova different, not in vague terms like simpler or more expressive, but in specific, structural terms. We compare Kova directly to the languages and tools it is most often compared to, and we explain why those comparisons miss the point.

3.1 The Central Difference

Most programming languages treat all values the same way at the type level. A string is a string whether it came from a database, a user input, a mathematical calculation, or an AI model. The language does not know the difference. It does not care.

Kova treats AI-produced values as a distinct type. They are not strings. They are not numbers. They are Prob<T>, a probabilistic wrapper that the language tracks separately from the moment of creation until the moment of explicit resolution. This distinction exists at the type system level, enforced before execution, with no way to bypass it.

No other general-purpose or domain-specific language does this. This is the central difference.

> **In one sentence:** *Other languages let you use AI outputs as if they were facts. Kova requires you to acknowledge they are estimates.*

3.2 Kova vs General-Purpose Languages

3.2.1 Python

Python is the dominant language for AI and machine learning work. It has excellent libraries for calling AI models: OpenAI's SDK, LangChain, Hugging Face's transformers, and many others. You can call an AI model in Python in a few lines.

But Python is dynamically typed. It has no concept of an AI-produced value. When an AI model returns a string to a Python program, that string is indistinguishable from any other string.

Python will not warn you if you use it carelessly. It will not reject your program if you put it directly into a condition without acknowledging its uncertainty.

```
# Python: AI output treated as a plain string
from openai import OpenAI
client = OpenAI()


response = client.chat.completions.create(
        model='gpt-4',
        messages=[{'role': 'user', 'content': 'Is this spam? ' + message}]
)
label = response.choices[0].message.content


# Python has no idea this came from an AI
# It is just a string
if label == 'spam':
        block_message()
```

Python is a better general-purpose language than Kova will ever be. It is not the right comparison. Python was not designed to enforce AI uncertainty. Kova was designed for exactly that.

3.2.2 JavaScript / TypeScript

JavaScript and TypeScript are the most common languages for backend web services, which is the domain Kova targets. TypeScript adds static types on top of JavaScript, which allows you to define custom types for AI outputs.

An experienced TypeScript developer might create a wrapper type to represent AI results. But this requires discipline. TypeScript will not force you to do it. It will not reject a program that uses a raw AI response without wrapping it. And even if you do wrap it, there is no built-in semantic equivalent of resolve() that the language enforces as the only path to unwrapping.

```
// TypeScript: you CAN wrap AI results, but nothing forces you to
type AIResult<T> = { value: T; confidence: number; model: string };

async function classify(text: string): Promise<AIResult<string>> {
    const result = await openai.chat.completions.create({ ... });
    return { value: result.choices[0].message.content, confidence: 0.85, model: 'gpt-4' };
}

// But nothing stops this:
const label = await openai.chat.completions.create({ ... });
if (label.choices[0].message.content === 'spam') { ... }
// TypeScript accepts this. Kova would reject it.
```

The difference is enforcement. TypeScript makes good patterns possible. Kova makes unsafe patterns impossible.

3.2.3 Rust

Rust has the most sophisticated type system of any mainstream language. Its ownership model and type checker catch entire categories of bugs at compile time that other languages only find at runtime. In principle, you could encode AI uncertainty in Rust's type system using a custom type and the type checker would enforce its usage.

But Rust is a systems programming language. Writing a backend AI workflow in Rust means managing memory explicitly, working with lifetimes, and fighting the borrow checker for operations that Kova handles in a few lines. The cognitive overhead is enormous for the use case Kova targets. And you would still be building the uncertainty tracking on top of Rust manually. It would not be part of the language's understanding of what AI calls are.

3.3 Kova vs AI Orchestration Frameworks

The most direct comparison most people reach for is not a programming language but an AI orchestration framework. LangChain is the most well-known. There are others: LlamaIndex, Haystack, CrewAI, and many more. These tools are worth examining carefully because they operate in the same space as Kova.

3.3.1 LangChain

LangChain is a Python and JavaScript library for building applications with large language models. It provides abstractions for chaining AI calls, managing prompts, connecting to vector databases, and building agents. It is genuinely useful and widely used.

But LangChain is a library, not a language. This is a fundamental difference. A library operates inside a host language and inherits all of that language's properties, including its inability to distinguish AI outputs from other values at the type level. LangChain can help you organise AI calls. It cannot enforce that you handle their outputs correctly, because it has no access to the host language's type system in the way a language designer does.

Kova is a language. It defines the type system. It defines what is valid. It can enforce things that a library operating inside another language cannot.

3.3.2 LlamaIndex and Similar Tools

LlamaIndex focuses on retrieval-augmented generation: connecting AI models to documents and data sources so they can answer questions about specific content. Like LangChain, it is a library with excellent tooling for its specific purpose. And like LangChain, it operates inside Python and inherits Python's type system, or lack thereof for AI uncertainty.

> **The key distinction:** *Frameworks and libraries make AI programming easier. Kova makes AI uncertainty structurally impossible to ignore. These are different goals at different levels of the stack.*

3.4 Kova vs Other DSLs

Domain-specific languages are the closest structural comparison to Kova. A few worth examining:

3.4.1 SQL

SQL is the most successful DSL in computing history. It is narrow: it does one thing, querying and manipulating relational data, and it does it well. Kova draws inspiration from SQL's approach to built-in domain primitives. Just as SQL has SELECT, INSERT, and UPDATE as first-class operations, Kova has GET, POST, find, insert, and AI as first-class operations.

SQL has no concept of AI uncertainty because SQL predates AI integration. But the design philosophy is similar: make the domain's core operations part of the language itself rather than leaving them to libraries.

3.4.2 GraphQL

GraphQL is a query language for APIs. It defines a type system for API responses and enforces that clients only request what they need. It is a good comparison for Kova's type enforcement philosophy: GraphQL makes it structurally impossible to access fields that do not exist in the schema. Kova makes it structurally impossible to use AI outputs without acknowledging their uncertainty.

3.4.3 Jsonnet / Dhall

Jsonnet and Dhall are configuration languages with type systems designed to prevent misconfiguration. They enforce structural constraints on data before it is used. Kova applies a similar philosophy to program execution: enforce structural constraints on AI-produced values before they influence control flow.

3.5 The Comparison Table

The following table summarises how Kova compares to the most relevant alternatives across the dimensions that matter for AI-integrated backend development:

Feature	Other Languages / Frameworks	Kova
AI uncertainty in type system	No. AI outputs are plain values.	Yes. AI() always returns Prob<T>.
Enforced uncertainty resolution	No. Optional at best.	Yes. resolve() is the only path.
AI calls as language primitives	No. Library functions only.	Yes. AI() is built into the language.
HTTP as language primitives	No. Requires libraries.	Yes. GET, POST, PUT, DELETE, PATCH.
DB operations as primitives	No. Requires ORMs or drivers.	Yes. connect, find, insert, update.
Execution graph output	No. Requires external tooling.	Yes. Every program produces one.
Parallel candidate detection	No. Manual analysis required.	Yes. Graph engine identifies candidates.

Semantic analysis of AI flow	No. Runtime only.	Yes. Checked before execution.
Purpose	General computation or broad tooling.	AI-integrated backend workflows.

3.6 What Kova Gives Up

Honesty requires acknowledging what Kova sacrifices for its specificity.

- Kova is not Turing complete in the practical sense of being able to express all programs. It lacks file I/O, multi-threading, native data structure libraries, and the full ecosystem of a general-purpose language.
- Kova does not have a package manager or an ecosystem of third-party libraries. What comes with the language is what you have.
- Kova's interpreter is slower than compiled languages. For performance-critical systems, this matters.
- Kova requires learning new syntax. Even if you know JavaScript, you are learning a new language when you learn Kova.

These are real limitations. They are the price of being a focused, research-oriented language rather than a production-ready platform. The goal of Kova is not to replace Python or JavaScript. It is to demonstrate a specific idea clearly enough that the idea can influence how those languages evolve.

3.7 The Broader Point: Languages Shape Thinking

There is a principle in linguistics called the Sapir-Whorf hypothesis: the language you speak shapes the thoughts you are able to think. The strong form of this claim is controversial. The weak form, that language influences how you frame problems, is widely accepted.

Programming languages work the same way. A language that has no type for uncertainty makes it easy to forget about uncertainty. A language that makes uncertainty visible at the type level makes it hard to forget. You see it in every variable name, every function call, every condition.

This is the deeper argument for Kova's approach. Even if every programmer understood AI uncertainty perfectly and handled it carefully, there would still be value in a language that makes the handling structural rather than discretionary. Structural constraints scale. Conventions do not.

When a team of ten engineers grows to a hundred, the conventions that worked at ten stop working at a hundred. Structural constraints enforced by the language work at any scale, because the language does not forget and it does not have bad days.

> **The argument:** *Languages shape how programmers think about problems. A language with a first-class probabilistic type shapes its programmers to think about AI uncertainty by default rather than as an afterthought.*

Next: Section 4 - Related Research

Section 4

Related Research

Kova did not emerge from a vacuum. It is situated at the intersection of multiple ongoing lines of research in programming languages, probabilistic programming, type systems, artificial intelligence safety, and neuro-symbolic computing. In this section, we focus on the work most closely related to what Kova does, connect it to our goals, and indicate how Kova differs from or builds on this previous work.

This section is for readers interested in understanding the context and related work behind Kova. While you don't need to have read this previous work to use Kova, understanding this context will help you appreciate our design decisions.

4.1 Probabilistic Programming Languages

The research area most directly adjacent to Kova is probabilistic programming. Probabilistic programming languages (PPLs) are languages designed to express probabilistic models: programs where variables represent probability distributions rather than fixed values, and where inference is a first-class operation.

4.1.1 Church and Anglican

Church is a PPL developed at MIT in the mid-2000s and is one of the first languages to support programming probability distributions as first-class values. That is, a Church program can specify a prior distribution for a variable, observe some evidence, and compute a posterior distribution. Church is built on top of Scheme and treats probabilistic inference as computation. Anglican is a PPL developed at Oxford that provides support for approximate probabilistic inference algorithms.

The connection with Kova is evident in the way Church and Kova both recognize the need to handle uncertain values as a distinct type. The difference is in scope and application. Church and Anglican are applicable to statistical modeling, building models of random processes. Kova is applicable to software development, building software that just happens to use AI models. In Kova, the uncertainty is not modeled; it is simply recognized and tracked.

4.1.2 Stan

Stan is a well-known PPL in statistics and data science. It is compiled to C++ and is used for defining and fitting Bayesian models. The type system of the Stan language has data, parameters, and transformed parameters as types. This is the foundation of the Kova Prob<T> type. Values of different origins and uncertainty characteristics should have different types. This is the foundation of the Kova type system. Stan applies this to statistical modeling; Kova applies this to AI software.

4.1.3 Pyro and NumPyro

Pyro is a probabilistic programming language developed by Uber AI, and NumPyro is the equivalent developed by DeepMind and Google. Both are probabilistic programming languages built on top of the PyTorch and JAX libraries, respectively. They are state-of-the-art in the field of probabilistic programming languages. They are libraries in the Python language and cannot enforce structural constraints on the language level. A program written in the Pyro language can make use of an AI model and the results of the model without any acknowledgment of the uncertainty of the AI model. This is what Kova addresses. The connection between the two is that probabilistic programming languages have already established the notion of uncertain values and the need for them to have different types.

> **Key connection:** *Probabilistic programming languages established the principle that uncertain values deserve distinct types. Kova applies this principle to a new context: not statistical modelling, but AI-integrated backend software.*

4.2 Type Systems and Effect Systems

The theoretical foundation of Kova's approach is in programming language theory, specifically in the study of type systems and effect systems.

4.2.1 Dependent Types

In a dependent type system, such as Agda, Coq, or Idris, types depend on values. This enables us to encode program properties directly into types, e.g., a function that returns a list whose length is equal to the length of the input list. Prob<T> is simpler than a dependent type, but similar in concept to it: more information is encoded into types to catch more errors at compile time. What is new and innovative about Kova is not the type encoding per se, but what it encodes: this is a value coming from a probabilistic source.

4.2.2 Effect Systems

In effect systems, types are extended to track what a program may do, what effects it may have, e.g., read from disk, write to a network, raise exceptions, or call probabilistic functions. Researchers have proposed languages such as Koka and Frank to support built-in effect systems. Prob<T> in Kova is similar to a lightweight effect system, where calling AI() introduces probabilistic effects, and this is tracked in the type of the result. An effect system for probabilistic effects of AI() is a natural extension, and this is mentioned in our future work section, 11.

4.2.3 Information Flow Types

In information flow type systems, type systems track information flow, i.e., where data comes from and where it may go. Typically, this is used for security applications. The fundamental papers in this area are Denning's lattice-based model and more recent languages such as Jif and FlowCaml. Our idea of removing a label to enable a value to be used in a context where a certain value is required is similar to information flow type systems. However, instead of security labels, we track uncertainty labels. The connection to information flow type systems is the deepest theoretical influence in our work.

Denning, D.E. and Denning, P.J. (1977). *Certification of programs for secure information flow.* Communications of the ACM, 20(7), pp.504-513.

4.3 Neuro-Symbolic Computing

Neuro-symbolic computing is a research area that seeks to combine neural networks, which learn from data and are good at perception and pattern recognition, with symbolic reasoning systems, which are good at logical inference and can be formally verified. The combination is motivated by the observation that neither approach alone is sufficient for general intelligence: neural networks are brittle under distribution shift, and symbolic systems struggle with noisy real-world inputs.

4.3.1 The Neuro-Symbolic Spectrum

Research in neuro-symbolic computing ranges from learning rules in a symbolic representation using gradient descent on data (Neural Theorem Provers, Differentiable Inductive Logic Programming) to using neural networks to generate proof steps that are verified by a symbolic procedure (AlphaGeometry). In between these two extremes are methods such as Neural Module Networks, in which inputs pass through a mixture of neural and symbolic components. Kova's approach occupies a particular place in this spectrum: it is a symbolic programming language that uses neural networks as external functions and tracks uncertainty in its type system. The resulting execution graph is a symbolic representation of where in the program the neural networks were invoked and how their outputs flowed through the symbolic representation of the program.

4.3.2 Symbolic Scaffolding for Neural Models

The idea of using symbolic information to aid in neural networks is also found in program synthesis, inductive programming, and neuro-symbolic concept learning. In these approaches, symbolic information is used to improve the quality of neural networks in terms of interpretability and robustness. Kova's execution graph is an example of such an approach: it is a symbolic representation of a program that uses neural networks. In theory, it could be used to improve neural networks or even verify them against some logic. For example, see Mao et al. (2019) for neuro-symbolic concept learning and Marcus (2019) for broader perspectives on AI robustness.

Mao, J., Gan, C., Kohli, P., Tenenbaum, J.B. and Wu, J. (2019). *The neuro-symbolic concept learner: Interpreting scenes, words, and sentences from natural supervision.* International Conference on Learning Representations (ICLR).

Marcus, G. (2019). *The next decade in AI: Four steps towards robust artificial intelligence.* arXiv:2002.06177.

4.4 AI Safety and Formal Verification

There is increasing research into making AI systems safe, verifiable, and alignable with human intent. Kova contributes to this area by making the uncertainty of AI structurally visible in the program, thus laying the groundwork for formal reasoning about AI-integrated systems.

4.4.1 Certified Machine Learning

In this area, researchers investigate the question of whether the behavior of the machine learning model can be formally guaranteed. Examples include robustness certifications, which ensure the output of the neural network will not change given certain perturbations in the input. The formal proofs are about the machine learning model. Kova, on the other hand, provides a guarantee about the program. A Kova program compiled to machine code is guaranteed to have dealt with the uncertainty of the AI explicitly. The machine learning model may not have produced the correct answer, but the program's handling of the uncertainty is correct.

4.4.2 Responsible AI and Auditability

Responsible AI has the goals of transparency, explainability, and auditability. Regulatory actions, such as the proposed EU AI Act, and guidelines issued by the US NIST and the UK AI Safety Institute increasingly call for the inclusion of audit trails for high-risk AI systems. Kova's execution graph is directly applicable to the problem. The Kova program will automatically generate a structural record of where the AI was called, what the task was, and how the result affected the program's execution. The inclusion of an execution graph is not an afterthought; it is part of the program's definition.

Doshi-Velez, F. and Kim, B. (2017). *Towards a rigorous science of interpretable machine learning.* arXiv:1702.08608.

4.5 Program Analysis and Execution Graphs

The execution graph Kova produces is related to a family of structures studied in program analysis.

4.5.1 Control Flow Graphs

The control flow graph (CFG) of a program is a directed graph whose nodes are the basic blocks of the program, and whose directed edges indicate the possible control flow between blocks. CFGs are the foundation of compiler optimization and static program bug detection tools. The extension of the execution graph of Kova adds domain-specific nodes like "ai," "resolve_prob," "http," and "db" to the control flow graph.

4.5.2 Data Flow Graphs

Data flow graphs represent the dependencies between program values, i.e., "a" depends on "b" if program A uses the value of program B. They are used for common subexpression detection and elimination, as well as in the ML community for implementing automatic differentiation algorithms. The graph of Kova combines control flow and data flow, including the explicit tracking of probabilistic flow (edges with Prob<T> are semantically distinct from those carrying specific data). This allows us to analyze the points at which uncertainty is introduced to and removed from the program.

4.5.3 Computation Graphs in Deep Learning

Deep learning frameworks like TensorFlow and PyTorch implement computation graphs to represent the computations of a neural network, whose nodes are the operations, and the directed edges represent the tensors flowing between operations, allowing for automatic differentiation and parallelization of the computation. The program graph of Kova, which is enabled by its AI capabilities, is similar to the computation graph of a deep learning framework, a directed graph of deterministic and probabilistic nodes. Therefore, a possible extension of the work of Kova could be the use of gradient-based optimization for the program's interactions with the AI.

Abadi, M. et al. (2016). *TensorFlow: A system for large-scale machine learning.* 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), pp.265-283.

4.6 Reinforcement Learning and Decision Making Under Uncertainty

Kova's research direction connects most directly to reinforcement learning (RL) and the broader study of decision making under uncertainty.

4.6.1 Markov Decision Processes

In MDPs, the agent perceives the state, takes an action, receives a reward, and then moves to the next state. The objective is to learn the mapping of states to actions, called the policy, to maximize the cumulative reward. A Kova program, in some sense, is similar to the concept of a policy, where the program perceives the states (input HTTP, database, AI), takes actions (responding, API calls, writing), and its computation is a sequence of states. The AI nodes identify the points where the decision is based on probability.

4.6.2 Partially Observable MDPs

POMDPs extend MDPs to situations where the agent cannot observe the true state of the world. Instead, they maintain a belief state (a probability distribution over possible states) and act based on this. This is similar to Kova, where invoking the function `AI()` to classify a piece of text is a partial observation in a partially observable world, where the true state is unknown, `AI()` provides probabilistic information about this unknown state, and invoking `resolve()` is to make a decision based on this probabilistic information about the unknown state. While this parallel is not yet formalized, this is one area for further research. If a Kova program can be viewed as a POMDP policy graph, then techniques for solving POMDPs could be used to analyze and optimize Kova programs.

Kaelbling, L.P., Littman, M.L. and Cassandra, A.R. (1998). *Planning and acting in partially observable stochastic domains.* Artificial Intelligence, 101(1-2), pp.99-134.

4.7 Domain-Specific Languages for AI Workflows

A small but growing body of work has explored domain-specific languages for AI and machine learning workflows specifically.

4.7.1 Halide

Halide is a DSL for image processing pipelines. It is designed to separate what to compute from how to compute it. Halide is important because it demonstrates that a carefully crafted and focused DSL can perform better than general-purpose programming languages for a particular domain. Similarly, Kova is a carefully crafted and focused language for a particular domain and derives all the advantages that come from being a domain-specific language.

4.7.2 ONNX and ML Interchange Formats

ONNX is a standard for representing machine learning programs to enable them to run across multiple frameworks. ONNX represents a machine learning program as a graph, where edges represent tensors and nodes represent operations. The execution graph in Kova, extended to capture all the semantics of AI-integrated software, could be used to standardize parallel software and integrate program logic and artificial intelligence interactions within a single auditable entity.

4.8 Where Kova Stands in This Landscape

Placed in this context, the role of Kova is quite evident. Kova is not a probabilistic programming language in the conventional sense, as it doesn't involve any Bayesian inference or Monte Carlo sampling. Kova is not a neuro-symbolic computing language, as it doesn't involve the learning of symbolic rules using neural representations. Kova is not a formal verification language, as it doesn't involve the formal proof of correctness of AI models. What Kova is, in precise terms, is a programming language that draws on the structural insights of these disciplines and applies them to the problem at hand. Kova draws on the insights of probabilistic programming in the structural notion of the use of different types for uncertain values. Kova draws on the insights of information flow typing in the structural notion of the use of the origins of values in the program. Kova draws on the insights of neuro-symbolic computing in the structural notion of the integration of symbolic and neural representations. Kova draws on the insights of reinforcement learning in the structural notion of the use of AI programs as policies in the context of uncertainty.

No paper or system has all these ideas in the precise form in which Kova represents them. This is what Kova contributes.

Summary: *Kova sits at the intersection of probabilistic programming, type theory, neuro-symbolic computing, and reinforcement learning. It is not a contribution to any one of these fields in isolation, but a synthesis of their core insights applied to a new problem: making AI uncertainty structurally visible in software.*

Next: Section 5 - Prerequisites

Section 5

Prerequisites

Everything you need to know before creating a language like Kova or comprehending its internal workings can be found in this section.

I have divided this section into two parts: what you need to know to use Kova, and what you need to know to build it or build on it.

I went over each idea starting with the first principle. You can skip the sections if you are familiar with them, but if you are not, start from 5.1 read through in order.

> **Important:** *You don't need to know everything in this section to start using Kova. Section 5.1 covers the minimum needed to write Kova programs. Section 5.2 onwards covers what you need to understand the internals.*

5.1 To Use Kova: Minimum Requirements

To write and run Kova programs, you need the following:

5.1.1 Basic Programming Concepts

Firstly, you need to know what a variable is. A variable is simply a named container.

For example:

```
let name = "Alice"
```

In the above example, we created a container called "name" and signed a value "Alice" of data type string into it. This is the same in Kova as Javascript, Python or any other language. I personally like to think of a variable as some kind of a labelled box that can hold a certain value of some certain data type.

Next, we need to know the answer to this question. What is a condition? A condition is a check or as I would love to say, a decision node that evaluates to true or false.

For example when you write:

```
if sentiment == "positive" {
        respond { status: 200, body: { approved: true } }
}
```

Here we are saying that if the value that variable sentiment holds is equal to “positive” , execute the expression the block embedded in the curly braces.

A step further, we need to understand what a function is in a program. A function is basically a named block of code that can be called with arguments and can also return a value. You definitely do not have to be a guru at functions but you need a solid understanding of the core idea.

5.1.2 What an API Is

Kova is designed for backend systems that call APIs. Now what is an API? Easy. An API is an acronym for Application Programming Interface, which is a way for two programs to interact with each other.

When you write this in Kova:

```
GET "https://api.example.com/users" into users
```

What it does is, it makes an HTTP request to a server at that address, asking to get data. The server then responds and the response is stored in the variable “users”. You do not have to explicitly know how to build an API to use Kova. You just have to understand what it means to call one.

5.1.3 What an AI Model Is (Loosely)

An AI model is a system that takes an input and produces an output. Here is the catch, the output is the best guess based on pattern recognized by the model from training data. So, it is not a calculation but an estimate.

For example, if you give an AI model the text “this product is terrible" and ask it to classify the sentiment, it might return negative. Most times, it would be right. It, but sometimes could be wrong. That uncertainty is the core thing Kova is built around.

5.1.4 Node.js Installed

Kova's interpreter runs on Node.js. You need Node.js version 18 or higher installed on your machine. You also need npm, which comes with Node.js, to install Kova and its dependencies.

```
# Check your Node.js version

node --version


# Should output v18.x.x or higher
```

5.2 To Understand the Kova Pipeline: Intermediate Requirements

Okay. This is the part I am most excited to share with you and I would love for you to pay some attention. If you want to understand how Kova works on the inside, and not just how to use it, then you need a broader foundation.

The following sub-sections explain each concept from scratch.

5.2.1 What a Compiler and Interpreter Are

When you write code in a programming language, the computer cannot run it directly. This is because the computer only understands machine code, ones and zeros. Machine code is basically binary instructions specific to its processor. Something has to convert your code into something the computer can execute.

There are two main approaches to this translation:

- A compiler translates i.e. converts the entire source code into machine code (or an intermediate form, It could be another high-level language but then it becomes a transpiler) before any of it runs. The output is an executable file. C and Rust use this approach.
- An interpreter reads the source code and executes it directly, line by line or statement by statement, without producing a separate executable. Python and Ruby use this approach. Kova uses this approach too.

The distinction matters because it strongly affects when errors are caught. A compiler catches errors before the program runs. An interpreter typically catches errors as it encounters them during execution. Kova's semantic analyzer, which runs before interpretation, moves some error detection earlier, which is one of its design features.

5.2.2 What Tokens Are

In the beginning, there were programs in their raw form. The program has to be processed for the machine to understand them and yield to the instructions. The first step is for them to be broken down into **tokens.** A token refers to merely the minimal meaningful unit of a language. I would prefer to consider it as words in the sentence. We will consider the following sentence, “the quick brown fox”. It has four tokens namely the, quick, brown, and fox.

Some examples of tokens in a programming language include keywords, identifiers, and operators like let to declare variables, if to control flow, userName and count, plus and equals,

the literal numbers like the number 42, some string like “hello”, and punctuation marks like curly braces and parentheses.

The sub-unit that does this tokenisation is known as a lexer or tokenizer. I recommend you call it “Lexer”, it is just cooler.

Kova has its own lexer written from scratch. When you write:

```
let x = 5 + 3
```

The lexer gives a sequence of tokens: LET, IDENTIFIER(x), EQUALS, NUMBER(5), PLUS, NUMBER(3). These tokens are processed by the rest of the pipeline, as opposed to the raw source text.

5.2.3 What an Abstract Syntax Tree Is

The second step after tokenising of the program is to comprehend its structure. A flat list of tokens fails to give the relationship of the parts of a program. An Abstract Syntax Tree (AST) does.

AST is a tree data structure corresponding to the structure of a program. The hierarchy of the code is reflected on this tree. The Program node will consist of the body or bodies, the body node will consist of functions, the function node will consist of a block, the block node will consist of statements, and the statement node consist of expressions, etc.

To write the expression 2+3*4, we would have a structure like this in the AST. The tree would have a parent node of binary expression of operation addition, with the left child to be a literal 2 and the right child to be a binary expression of operation multiplication. On this node, on the left we would have literal 3 and on the right we would have literal 4. This order is operator precedence as it should be: multiplication is a more specific operator than addition, and therefore the 3 times 4 expression is computed first, then the result is computed in the addition expression.

The component that constructs the AST from the tokens is referred to as a parser. Kova uses a type of parser known as the recursive descent parser, which is a top-down parser with no back-tracking. This type of parsing is one of the most understandable and readable parsing methods. The rules of grammar of the language are each translated into a grammar rule in the parser.

5.2.4 What Semantic Analysis Is

Parsing tells you whether a program is syntactically valid. It checks whether the program follows the grammar rules of the language. But a program can be syntactically valid and still be meaningless.

For example:

```
let x = y + 1   // y is not declared
```

This is syntactically valid: it follows all the grammar rules. But it is semantically invalid: y does not exist. Semantic analysis catches this class of error.

Kova's semantic analyzer does several things. It checks that all variables are declared before use. It checks that functions are called with the right number of arguments. And most importantly for Kova's design, it checks that Prob<T> values are never used in contexts that require a certain value. This last check is what makes Kova's AI uncertainty enforcement structural rather than optional.

5.2.5 What Interpretation Is

After the AST has been built by the parser and semantically validated by the semantic analyzer, the interpreter executes it. Kova uses a type of interpreter known as tree-walk interpreter. This is the simplest form of interpreter. What it does is walk the AST recursively, visiting each node and simply executing the operation it represents.

In visiting a binary expression such as 3 plus 4 it recursively evaluates the operands and returns their sum. Once it reaches an if statement node it will analyze the condition and follow the corresponding branch.

It is worth noting that the time taken by tree- walk interpreters is more than compiled code since re-traversing the tree is necessary each time they execute a loop, but they are easy to understand and easy to extend, which makes them the right choice for a research language like Kova.

5.2.6 What a Type System Is

A type system describes a set of rules that assigns type to all values and expressions of a program and ensures that all operations are performed on values of the right type. Type systems eliminate a class of errors whereby you attempt to perform some operation on a value that does not fit its type, such as adding a string and a boolean.

Different languages have different type systems. Some check types at compile time (static typing). Some check at runtime (dynamic typing). Some allow you to mix both.

Kova is mostly dynamically typed: you do not declare the type of variables. But Prob<T> values are statically checked: the semantic analyzer verifies at analysis time, before execution, that they are never used without resolution. This hybrid approach gives Kova flexibility for general code while providing strict guarantees where AI uncertainty is involved.

5.2.7 What a Graph Is (in Computer Science)

A graph is a data structure, consisting of nodes and edges. Edges and nodes are represented by things and relationships between things respectively. In a directed graph, the edges are those that are directional, i.e. from node A to node B, not necessarily back from B to A.

Kova's execution graph is a directed graph. Each operation in the program is a node. Each dependency between operations is a directed edge. If operation B uses a value produced by operation A, there is a directed edge from A to B.

Graphs have a rich theory. Two properties of Kova's execution graph that are particularly useful are:

- Topological ordering: a way of listing the nodes so that every node comes after all the nodes it depends on. This gives you a valid execution order for the program.
- Parallel candidates: pairs of nodes that have no dependency relationship, meaning they could be executed simultaneously without affecting the result.

5.3 To Build or Extend Kova: Advanced Requirements

For you to add new features i.e becoming a contributor on the kova codebase or for you to build a new language from scratch, I think we both agree that you need a deeper foundation in addition to everything we have discussed prior.

5.3.1 JavaScript and Node.js (Intermediate Level)

Kova is built with Javascript running on Node.js.You need to be comfortable coding with modern javascript. This includes understanding ES modules, async / await, destructuring, spread operators, and classes. It is worthy to note that you do not need to be an expert, but you need to be able to read and write JavaScript fluently.

Specifically relevant to Kova's codebase:

- Recursive functions: the parser and interpreter both use heavy recursion.
- Map and Set data structures: used throughout the interpreter and graph engine.
- Closures: essential for understanding how Kova implements lexical scope.
- Async/await: used in the async interpreter for handling Groq API calls.

5.3.2 Formal Grammars

A formal grammar is some form of rules that defines the syntax of a language. The most common notation for programming language grammars is Backus-Naur Form (BNF) or Extended BNF (EBNF).

A grammar rule like:

```
expression := term (('+' | '-') term)*

term := factor (('*' | '/') factor)*

factor := NUMBER | '(' expression ')'
```

Defines a language for arithmetic expressions with correct operator precedence. Each rule defines one grammar construct in terms of other constructs or terminals (literal characters). Kova's parser implements a grammar like this, with one function per grammar rule.

You do not need to know formal language theory deeply to work on Kova, but you should be able to read a grammar specification and map it to parsing code.

5.3.3 Tree Data Structures

The AST is a tree. Working with it requires comfort with tree data structures and recursive algorithms. The main operations you need to understand and be able to do are:

- Building a tree by constructing nodes and linking them as children of other nodes.
- Traversing a tree recursively, i.e. visiting a node, then recursively visiting its children.
- The visitor pattern. This is a way of organising tree traversal so that different operations on the tree are kept separate from the tree structure itself.

5.3.4 Scope and Symbol Tables

Scope in a program defines the limit of a variable's visibility. In most languages, a variable declared in a function is invincible to the outside world of the function. When a variable is

referenced, the interpreter must find which declaration it refers to by searching up through the chain of enclosing scopes.

We implement a scope using a scope stack. This is a stack of symbol tables where each symbol table maps variable names to their values or types. You enter a block, then you push a new symbol into the stack and when you leave you pop it out. When you look up a variable, you search from the top of the stack downward. This is similar to using the stack method to traverse a tree using the Depth First Search (DFS) approach.

Kova implements this for both the semantic analyzer (which tracks types) and the interpreter (which tracks values). Understanding scope is essential for working on either.

5.3.5 Graph Algorithms

Kova's execution graph engine uses several graph algorithms. You do not need to implement them from scratch, but you should understand what they do:

- Depth-first search (DFS): a traversal algorithm that explores as far as possible along each branch before backtracking. Used in topological sorting.
- Topological sort: an algorithm that produces a linear ordering of nodes in a directed acyclic graph such that every node comes before all nodes it has edges to. Used to determine a valid execution order.
- Cycle detection: an algorithm that determines whether a directed graph contains a cycle. Important because a cycle in the execution graph would mean the program has a circular dependency.

5.4 Recommended Learning Resources

If you are lacking any of the above prerequisites, here are the best ways to learn what you are missing.

5.4.1 For Programming Language Implementation

- Crafting Interpreters by Robert Nystrom (craftinginterpreters.com). This is hands down the best free resource available for building an entire interpreter from scratch. It takes you

through an entire implementation of a language in java and C, taking time to explain each concept. Read nothing before seeing the source code of Kova but this.

- Writing an interpreter in Go by Thorsten Ball. Short, practical and builds up a working interpreter step-by-step. Good alternative if you like your books short.
- Structure and Interpretation of Computer Programs (SICP) by Abelson and Sussman. The classic text when it comes to language semantics and building interpreters, thick and authoritative.

5.4.2 For Type Systems

- Types and Programming Languages by Benjamin Pierce. The most definitive textbook in the field. You only need to read chapters 1-10 for this task.
- Practical Foundations of Mathematics by Paul Taylor. More complex but has much better explanations for those who want the mathematical background of type theory.

5.4.3 For Graphs and Algorithms

- Introduction to Algorithms by Cormen, Leiserson, Rivest, and Stein (CLRS). The defacto standard textbook covering the area. The sections regarding DFS, Tarjan's algorithm for SCCs and topological sort should be most helpful.
- Algorithms by Jeff Erickson (jeffe.cs.illinois.edu). Online, freely available, also well-written with good explanations regarding graphs.

5.4.4 For JavaScript

- JavaScript: The Definitive Guide by David Flanagan. The most comprehensive guide to javascript.
- The Node.js documentation (nodejs.org). You need to be comfortable with the environment that Kova runs in.
- MDN Web Docs (developer.mozilla.org). Use it as a quick reference when needed.

5.5 What You Do Not Need

It is worth being explicit about what you do not need, to avoid discouraging people who could contribute to or use Kova.

- You do not need a computer science degree. The concepts in this section can be learned from the resources above without any formal education.
- You do not need to know machine learning or deep learning to use or understand Kova. Kova treats AI models as black boxes. What matters is the type of the output, not the internal mechanism of the model.
- You do not need to know category theory, lambda calculus, or formal language theory at a deep level to work on Kova. These are relevant to the theoretical foundations, but Kova's implementation is practical and accessible.
- You do not need prior experience with language implementation. If you have read any of the recommended texts for learning interpreters, you have enough to understand Kova's codebase.

Bottom line: *You understand recursive functions and understand what a tree is? Then you are good to start. You will learn everything else along the way*

Next: Section 6 - How Kova Works: The Full Pipeline

How Kova Works: The Full Pipeline

This section details exactly how Kova will take a text file and execute it. Each stage in the pipeline is broken down to show the journey from the first character the lexer reads, to the execution graph the program generates. Each aspect of this section uses a snippet from Kova's source code.

The pipeline has seven stages:

1. **Lexer** Reads raw source text. Produces a flat list of tokens.
2. **Parser** Reads tokens. Produces an Abstract Syntax Tree (AST).
3. **AST** The tree structure that represents the program's meaning.
4. **Semantic Analyzer** Reads the AST. Validates types, scope, and Prob<T> rules.
5. **Interpreter** Walks the AST. Executes each node. Produces output and return values.
6. **Execution Graph** Built from the AST. Captures control flow, data flow, and AI node positions.
7. **AI Layer** Handles Prob<T> creation and resolution. Connects to Groq or uses stubs.

6.1 The Lexer

6.1.1 What It Does

The lexer is the very first step of the pipeline. It takes raw source code from the file (as a string of characters) and breaks it down into a flat list of tokens (also called lexemes). This is the process of tokenisation, or lexical analysis.

As an analogy, the lexer is a reader who is not familiar with Kova, but just knows the alphabet. It reads the source code from left to right, and groups the characters to chunks, assigns a 'type' to

each chunk, and doesn't know the meaning of a chunk, nor the order of chunks, nor the way to handle chunks based on other chunks (all of these jobs are later stages of the pipeline).

```
// Source code fed into the lexer
let x = 5 + 3




// What the lexer produces
[ { type: 'LET',          value: 'let', line: 1, col: 1 },
 { type: 'IDENTIFIER', value: 'x',   line: 1, col: 5 },
 { type: 'EQUALS',       value: '=',   line: 1, col: 7 },
 { type: 'NUMBER',       value: 5,          line: 1, col: 9 },
  { type: 'PLUS',          value: '+',   line: 1, col: 11 },
 { type: 'NUMBER',       value: 3,          line: 1, col: 13 } ]
```

Token stream produced by the Kova lexer

There are four values stored by each token: type, value, line and column. Line and column are line number and position of the character on the line respectively, and they are there to give the developer helpful information in case of errors in their program-the error reporting functionality of Kova will be able to tell the developer the exact location of the problem.

6.1.2 How the Kova Lexer Works

Kova lexer (src/lexer/lexer.js) inherits the class (src/lexer/base.js), which is designed to perform elementary tokenization operations:

```
// From src/lexer/lexer.js

export default class Lexer {

        constructor(code) {

        this.code            = code || "";

        this.position        = 0;

        this.currentChar = this.code.length ? this.code[0] : null;

        this.line            = 1;

        this.column          = 1;

        }



        advance() {

        if (this.currentChar === "\n") {

        this.line++;

        this.column = 1;

        } else {

        this.column++;

        }

        this.position++;

        this.currentChar = this.position < this.code.length

        ? this.code[this.position]

        : null;

        }
```

```
        peek(offset = 1) {

        const pos = this.position + offset;

        return pos < this.code.length ? this.code[pos] : null;

        }

}
```

Lexer class constructor and core movement methods

6.1.3 Skipping Ignorable Characters

The Kova lexer calls skipIgnorable() just before it attempts to read each token. The purpose of this function is to read over any parts of the code which should not be part of the syntax of the program-namely, whitespace and comments.

6.1.4 Reading Different Token Types

When it's not skipping ignored characters, the lexer tries to identify a valid token. Each token type is handled differently, and the follwoing functions are responsible for them.

- readNumber() : This function ensures that Numbers can be integers or floats (e.g. 42, 3.14) but cannot start with a letter (e.g. 2x is invalid) and store the token in the form {type: "NUMBER", value: Number(number), float: isFloat, line, column}
- readString() : This function ensures String literals can be enclosed in single or double quotes, and support common escape sequences and store the token in the form {type: "IDENTIFIER", value: text, line, column }.
- readIdentifierOrKeyword(): This function checks for Identifiers and keywords (e.g. if, else, while) or user-defined names (e.g. myVar).
- readOperator(): Check for multi-character operators first (e.g. ==, !=, <=, >=), then single-character ones.
- readProgramSymbols(): Checks for single-character program symbols like parentheses, commas, etc.

- tokenize(): The main function that gets called to handle all the lexer operation.

> **Key insight:** *The lexer does not understand what the tokens mean. It only recognises patterns of characters. Understanding comes later, in the parser and semantic analyzer.*

6.1.5 What the Lexer Produces

The lexer returns an array of token objects. This array represents the sequence of tokens that makes up the source code. It will be passed as the sole input to the parser.

6.2 The Parser

6.2.1 What It Does

The parser then uses the flat list of tokens provided by the lexer, to build an Abstract Syntax Tree. The lexer understands characters; the parser understands grammar; how tokens form expressions, expressions form statements, statements form programs.

The Kova parser is implemented as a recursive descent parser. It is an ancient and human readable approach. Each of the grammar rules has a separate function. The functions take tokens, make recursive calls if needed and return an AST node.

6.2.2 The Entry Point

The entry point for the parser is parseProgram(). The function iterates through all the tokens and repeatedly calls parseStatement() until all tokens are processed. ParseStatement() inspects the current token type, and then call on to one of the parsing functions:

```
// From src/parser/parser.js

parseStatement() {

        while (this.currentToken.type === "SEMICOLON") this.advance();
```

```
        switch (this.currentToken.type) {
        case "LET":       return this.parseVariableDeclaration();
        case "FN":        return this.parseFunctionDeclaration();
        case "IF":        return this.parseIfStatement();
        case "WHILE":  return this.parseWhileStatement();
        case "FOR":       return this.parseForStatement();
        case "RETURN":          return this.parseReturnStatement();
        case "RESPOND":         return this.parseRespondStatement();
        case "CONNECT":         return this.parseConnectStatement();
        case "FIND":      return this.parseFindStatement();
        case "INSERT":  return this.parseInsertStatement();
        case "UPDATE":          return this.parseUpdateStatement();
        case "GET_HTTP":
        case "POST_HTTP":
        case "PUT_HTTP":
        case "PATCH_HTTP":
        case "DELETE_HTTP": return this.parseHttpStatement();
        default:          return this.parseExpressionStatement();
        }
}
```

The parser dispatcher, each token type routes to its own parsing function

6.2.3 Expression Parsing and Precedence

We also need to take operator precedence into account for parsing an expression. It's important that "2+3*4" is parsed as 2+(3*4) rather than (2+3)*4. Recursive descent does this automatically by having a parsing function for each precedence level.

```
// Precedence hierarchy in the Kova parser (simplified)
parseExpression()        // lowest: assignment (=, +=, -=)
  parseLogicalOr()       // ||
  parseLogicalAnd()   // &&
  parseEquality()        // ==, !=
  parseComparison()   // <, >, <=, >=
  parseAdditive()        // +, -
  parseMultiplicative() // *, /, %
  parseUnary()           // !, -
  parsePostfix()         // member access, function calls, index
  parsePrimary()         // literals, identifiers, grouped exprs
```

Expression parsing hierarchy - lower functions bind more tightly

Each level calls the next level down for its operands. This naturally produces a tree where higher-precedence operations are deeper in the tree and evaluated first.

```
// A find statement
find users where { active: true } limit 10 into results


// The AST node it produces
{
  type:    "FindStatement",
  collection: "users",
  where:              { type: "ObjectLiteral", properties: [{ key: "active", value: true }] },
  limit:   10,
  orderBy:            null,
  binding:            "results"
}
```

How a find statement becomes an AST node

6.2.4 Error Handling in the Parser

If the parser is presented with an unexpected token then it calls expect().This function will print a message saying which token the parser would have expected at the location of the illegal token and what line and column number the illegal token appeared. Syntax errors in Kova source will thus show the correct location of the source.

6.3 The Abstract Syntax Tree

6.3.1 What It Is AST?

The AST is the main data structure in the pipeline. It is a tree composed of JavaScript objects. Each object is a node of the tree and represents some construct of the program. When Kova processes an entire program, it generates one single tree rooted on a Program node.

The tree is called Abstract as it does not contain every feature of the program from the text representation of source code, the parenthesis, semicolons, Whitespace have been removed in order to emphasize the structural representation.

6.3.2 AST Node Structure

Every AST node is a normal JavaScript object that has a "type" field describing the kind of node it is. All further fields will describe the data in the specific kind of node, and there are a number of these:

```
// let x = 5
{ type: "VariableDeclaration",
  id:   { type: "Identifier", name: "x" },
  init: { type: "NumericLiteral", value: 5 } }


// if sentiment == "positive" { ... }
{ type: "IfStatement",
  test: { type: "BinaryExpression",
          operator: "==",
          left:  { type: "Identifier", name: "sentiment" },
          right: { type: "StringLiteral", value: "positive" } },
  consequent: { type: "BlockStatement", body: [...] },
```

```
 alternate:  null }



// let p = AI("classify", text)

{ type: "VariableDeclaration",

 id:   { type: "Identifier", name: "p" },

 init: { type: "CallExpression",

          callee: { type: "Identifier", name: "AI" },

          arguments: [

          { type: "StringLiteral", value: "classify" },

          { type: "Identifier",      name: "text" }

          ] } }
```

Example AST nodes from Kova programs

6.3.3 Why the AST Matters

The AST is what the semantic analyzer and interpreter will use. The semantic analyzer will walk it once, reporting errors, and the interpreter will walk it once more to execute nodes. The execution graph engine will also walk it, constructing the graph.

Since the AST is composed of pure JavaScript objects (which forms a tree structure), it can be easily examined, printed out, or reasoned about. From src/index.js, the parseKova() function exports the AST directly, useful for tooling, debugging and reasoning.

The key insight: The AST is a contract. The parser has the job of creating a correct AST. The remainder of the pipeline has the job of interpreting it.

Key insight: *The AST is the contract between the parser and everything that comes after it. The parser's job is to produce a correct AST. The rest of the pipeline's job is to consume it.*

6.4 The Semantic Analyzer

6.4.1 What It Does

The semantic analyzer walks through the AST and checks if the program means what it says it means. It actually does not execute the program at all. Instead it looks for errors that the parser will not be able to detect since those kinds of errors depend on the meaning of a certain code block, not on the syntax.

There are three main kinds of check that Kova's semantic analyzer makes:

6.4.2 Scope Implementation

The semantic analyzer keeps a scope stack: an array of Maps each of which is a symbol table for a single lexical scope. When the semantic analyzer enters a block (such as a function body, an if body, a loop body), it pushes a Map onto the stack. When it leaves the block, it pops the Map.

When a variable is declared, it is placed in the current (topmost) scope. When a variable is referenced, the semantic analyzer searches the stack starting from the top and going down, and returns the first scope it finds. If it doesn't find the variable it throws a Diagnostic error with the location of the reference.

```
// From src/semantic/semanticAnalyzer.js

enterScope() { this.scopes.push(new Map()); }

exitScope()  { this.scopes.pop(); }
```

```
declare(name, type) {
        const scope = this.scopes[this.scopes.length - 1];
        if (scope.has(name))
        throw new Error(`Variable "${name}" already declared in this scope`);
        scope.set(name, type);
}


resolve(name, node = null) {
        for (let i = this.scopes.length - 1; i >= 0; i--) {
        if (this.scopes[i].has(name)) return this.scopes[i].get(name);
        }
        if (Object.prototype.hasOwnProperty.call(this.externals, name))
        return "external";
        if (this.functions[name]) return "function";
        throw new Diagnostic(`Undeclared variable "${name}"`, node, this.source);
}
```

Scope stack implementation in the semantic analyzer

6.4.3 The Prob<T> Type System

The Prob<T> type is represented internally as a JavaScript object: { kind: "prob", inner: T }. The inner field carries the type that the AI output would have after resolution. Three helper methods on the analyzer handle all Prob<T> operations:

```
// From src/semantic/semanticAnalyzer.js
```

```
Prob(inner)   { return { kind: "prob", inner }; }

isProb(t)            { return t && typeof t === "object" && t.kind === "prob"; }

unwrapProb(t) { return t.inner; }
```

The three Prob<T> helpers used throughout the semantic analyzer

If the analyzer encounters a CallExpression whose callee is AI then this.Prob(sig.returns) is returned instead of the bare return type. This annotates the return type as a Prob<T> marker. The type of the assigned variable will then be recorded as { kind: "prob", inner: "unknown" } in the scope.

If the analyzer encounters a binary operation, if-condition, or function argument, it will check if any of the required types are Prob<T> (using isProb). If so, then it throws an error immediately:

Whenever the analyzer visits a binary expression, an if condition, or a function argument, it checks whether any of the relevant types are Prob<T> using isProb(). If they are, it throws an error immediately:

```
// What happens when you try to use a Prob<T> in a condition

let probSentiment = AI("classify", text)



// This will be rejected by the semantic analyzer:

if probSentiment == "positive" { ... }



// Error output:

// KovaSemanticError: Equality operands must be the same type, got prob and string

// --> Line 4, Column 18
```

```
//          4 | if probSentiment == "positive" {

//                    ^
```

The semantic analyzer rejecting a Prob<T> used without resolution

The only way to extract a value from Prob<T> is to call resolve(). When the analyzer sees resolve(probSentiment), it calls unwrapProb() on the argument's type and returns the inner type. The variable that receives the resolved value is typed as the inner type, not as Prob<T>. From that point on it can be used in any context.

6.4.4 Error Reporting

Errors from the semantic analyzer are instances of the Diagnostic class, defined in src/core/diagnostic.js. A Diagnostic carries the error message, the AST node where the error occurred, and the original source code. Its format() method produces the formatted error output with the source line and caret pointer that Kova displays to the user.

```
// Example semantic error output

KovaSemanticError: Undeclared variable "userName"

--> Line 3, Column 12



  3 | let greeting = userName + " welcome"

                    ^
```

Formatted diagnostic output from the semantic analyzer

6.5 The Interpreter

6.5.1 What It Does

The interpreter then takes the validated AST and "walks" through it, performing the action corresponding to each node in the tree recursively. The Kova program returns a Javascript object consisting of the returned value, the response value and any printed output gathered.

Kova has two interpreter modes: async and sync. The async interpreter supports real AI calls via Groq. The sync interpreter uses stubs only and is used for testing and offline development. Both modes share the same visitor logic; the difference is whether the visitor functions are async and awaited.

6.5.2 Interpreter State

The interpreter maintains several pieces of state throughout execution. These are stored as instance variables on the Interpreter class:

```
// From src/interpreter/interpreter.js

export default class Interpreter {

        constructor(externals = {}) {

        this.scopes         = [];      // scope stack for variable values

        this.output         = [];      // collected print() output

        this.externals      = externals; // injected functions (AI, resolve, etc.)

        this.returnValue  = undefined; // set when return statement executes

        this.shouldReturn = false;             // flag to unwind the call stack

        this.functions      = {};      // user-defined function registry

        this.connections  = {};      // active DB connections

        this.respondValue = undefined; // set when respond statement executes

        }
```

```
}
```

Interpreter state fields

The scopes stack works the same way as in the semantic analyzer: a stack of Maps where each Map holds variable name to value bindings for one lexical scope. The difference is that here the values are actual runtime values, not type strings.

The shouldReturn flag is how Kova handles early returns inside loops and nested blocks. In case of returning something the shouldReturn flag is set to true and the return value is stored. In the beginning of every visitor function the flag is checked and returned if it's set, recursively unwinding the function call stack until the function boundary.

6.5.3 The Visit Method

The central part of the interpreter is the visit() method that uses a switch statement on the node type and calls the relevant visitor method. Each type of AST node should have its own visitor method, to call. Here is how several important node types are handled:

```
// VariableDeclaration: let x = 5 + 3
case "VariableDeclaration": {
        const value = await this.visit(node.init);
        this.declare(node.id.name, value);
        return value;
}


// BinaryExpression: 5 + 3, x == "positive", etc.
case "BinaryExpression": {
        const left  = await this.visit(node.left);
```

```
        const right = await this.visit(node.right);
        switch (node.operator) {
        case "+":  return left + right;
        case "-":  return left - right;
        case "*":  return left * right;
        case "/":  if (right === 0) throw new RuntimeError("Division by zero");
                return left / right;
        case "==": return left === right;
        case "!=": return left !== right;
        case "<":  return left < right;
        case ">":  return left > right;
        }
}

// IfStatement: if condition { ... } else { ... }
case "IfStatement": {
        const cond = await this.visit(node.test);
        if (cond) {
        await this.visit(node.consequent);
        } else if (node.alternate) {
        await this.visit(node.alternate);
        }
        return undefined;
```

```
}
```

Selected visitor cases from the interpreter

6.5.4 Built-in Functions

The interpreter has a _builtins object that holds all built-in functions. These are functions available in every Kova program without importing anything. They include print, len, typeOf, range, toString, and all the standard math functions. When a CallExpression is visited, the interpreter first checks _builtins, then the externals injected at construction time, then user-defined functions.

The typeOf built-in is worth noting specifically. It returns "prob" for Prob<T> values, not "object" as JavaScript's own typeof would. This is implemented by checking the isProb() function from the AI layer before falling back to JavaScript's typeof:

```
typeOf: (v) => {
        if (v === null)       return "null";
        if (Array.isArray(v))  return "array";
        if (isProb(v))        return "prob";
        return typeof v;
},
```

typeOf built-in - aware of the Prob<T> type

6.5.5 HTTP, Database, and Respond Handling

Kova's domain-specific statements are handled directly in the interpreter. They are not compiled to library calls or translated into something else. The interpreter handles them as first-class execution steps.

HTTP statements produce a stub response object containing ok: true, status: 200, and any body provided. In a production runtime, these would make real HTTP requests. The current implementation outputs a log line and returns the stub, which is sufficient for testing program logic. The into binding stores the response in a variable.

Database operations work similarly. connect stores a connection object keyed by driver name. find, insert, and update produce result stub objects with properties like collection, insertedId, modifiedCount, and limit that the program can read. This allows realistic backend workflows to be written and tested without a real database.

The respond statement sets this.respondValue on the interpreter and sets shouldReturn = true, which unwinds execution. The respond value is included in the result object returned by interpret().

6.5.6 Function Calls and Closures

Kova user-defined functions support closures: function can refer variables not only in its call time scope but also its definition time scope. This is done by capture the current scope chain at function definition time and restore it when function is called.

When a FunctionDeclaration is visited, the interpreter would store the function definition(param list, body, and the current scope chain) in this.functions. When a CallExpression calls the function, the interpreter will restore the captured scope, push the new scope for function local variables and bind args with params, and visit the body and then pop out the scope and restore caller's scope.

6.6 The Execution Graph

6.6.1 What It Does

The graph representing the Kova program is always constructed and returned when running through runKova() and runKovaSync().

It comprises 3 types of information, regarding the actions (nodes) performed by the program, the way they interact each other (edges representing the sequence and the control flows between them), and which of them are using an input produced by another one (data edges).

6.6.2 Node Kinds

Every meaningful operation in a Kova program becomes a graph node. The full set of node kinds, taken directly from the source, is:

```
entry    - program start
exit     -  program end
declare  variable declaration      (let x = ...)
assign   variable reassignment     (x = ...)
http     - HTTP call               (GET / POST / PUT / DELETE / PATCH)
db_connect   - database connection      (connect ... using ...)
db_find      - database read            (find ... where ...)
db_insert    - database write           (insert into ...)
db_update    - database mutation        (update ... set ...)
respond      - respond statement
call     - function call
fn_def   - function declaration
if       - conditional branch
while    - while loop
for      - for..in loop
return   - return statement
ai       - AI() call               (produces Prob<T>)
```

```
resolve_prob - resolve() call        (unwraps Prob<T>)
```

All node kinds in the Kova execution graph

The ai and resolve_prob node kinds are unique to Kova. They mark precisely where uncertainty enters the program and where it is explicitly resolved. A reviewer reading the graph output can immediately see how many AI calls were made, whether every one was resolved, and in what order resolution happened relative to other operations.

6.6.3 Edge Kinds

Edges connect nodes and carry a kind label that describes the nature of the relationship:

- **seq:** sequential execution. Node B runs after node A.
- **data:** data dependency. Node B reads a value produced by node A. The edge label is the variable name.
- **control:** conditional dependency. Node B runs only if node A's condition evaluated to true or false. The edge label is 'true' or 'false'.
- **call:** function invocation. A call node is connected to the fn_def node of the function it calls.
- **binding:** result binding. When an operation uses into <var>, a binding edge connects the operation to the declaration node for that variable.

6.6.4 How the Graph Is Built

Graph Engine is implemented in src/graph/executionGraph.js. It defines ExecutionGraph class, which stores a Map from node id to its AST node and an array of edges. Build() walks AST and emits a node and its incoming/outgoing edges for each encountered statement or expression.

We track variables using _varProducers Map: a Map from varname to node ID of the last statement writing to it. When a node depends on a variable, graph engine find producer of the var and add a data edge. This allows data dependency edges of graph to be correct.

```
// From src/graph/executionGraph.js

_setProducer(name, nodeId) { this._varProducers.set(name, nodeId); }

_getProducer(name)        { return this._varProducers.get(name) ?? null; }



// Add data-dependency edges from all vars referenced in astNode to nodeId

_dataDeps(astNode, nodeId) {

        this._refsIn(astNode).forEach(name => {

        const producer = this._getProducer(name);

        if (producer) this.addEdge(producer, nodeId, "data", name);

        });

}
```

Variable tracking and data dependency edge creation

6.6.5 Topological Ordering

Once the graph is constructed, the graph engine calculate the topological order: a linear ordering of all the nodes such that each node appears after all nodes it depends on. It provides an executable order for the program. It is also helpful for giving the logical order of the program at first sight.

Topological ordering is only possible if the graph has no cycles. A cycle would mean operation A depends on B which depends on A, which is logically impossible. Kova's graph engine detects cycles during the topological sort and would throw an error if one were found, though well-formed Kova programs do not produce cyclic graphs.

6.6.6 Parallel Candidates

The graph engine also identifies parallel candidates: pairs of nodes that have no dependency relationship between them, either direct or transitive. Two such nodes could in principle be executed simultaneously without affecting the result.

This is particularly meaningful for AI calls. In a program that makes three independent AI() calls, the graph engine identifies all three as parallel candidates, meaning a future Kova runtime could send all three requests to Groq simultaneously rather than waiting for each to complete before starting the next. The graph makes this optimisation visible and computable.

```
// Three AI calls with no dependencies between them
let probCategory  = AI("classify category", article)
let probSummary   = AI("summarize",       article)
let probSentiment = AI("classify sentiment", article)


// The graph identifies these as parallel candidates:
// Parallel candidates: n13 || n14, n13 || n15, n14 || n15
// All three AI nodes can run concurrently
```

Parallel AI calls identified by the execution graph engine

6.6.7 Graph Output Format

The graph is returned from runKova() as part of the result object. It has several properties:

- graph.json.nodes: array of all nodes with id, kind, label, and meta fields
- graph.json.edges: array of all edges with from, to, kind, and label fields
- graph.topologicalOrder: array of node IDs in topological order
- graph.sourceNodes: nodes with no incoming edges, the starting points of execution
- graph.parallelCandidates: array of pairs of node IDs that have no dependency relationship

6.7 The AI Layer and Prob<T>

6.7.1 Overview

The AI layer is in src/ai/groq.js. It has four exports: makeProb, isProb, resolveProb, and groqAI. There is also stubAI for offline use. Together, these five functions implement the entire probabilistic type system at runtime.

6.7.2 The Prob<T> Object

At runtime, a Prob<T> value is a plain JavaScript object with six fields:

```
// The Prob<T> object structure
{
    __prob__:   true,          // marker field — identifies this as Prob<T>
    value:    T,          // the actual AI output (string, object, etc.)
    confidence: number,        // 0.0 to 1.0 — model's confidence estimate
    model:   string,    // which model produced this value
    task:      string,    // the task string passed to AI()
    tokens:  number,           // token count from the API response
    resolved:   false,         // set to true when resolve() is called
}
```

The Prob<T> runtime object: a plain JavaScript object with a marker field

The __prob__ field is the marker that isProb() checks. It is chosen to be unlikely to conflict with normal object keys. The interpreter's typeOf built-in uses isProb() to return "prob" for these objects instead of "object".

6.7.3 makeProb and stubAI

The makeProb() function creates a new Prob<T> object from a value and optional metadata. stubAI() calls makeProb() with a placeholder value string that includes the task name, so you can see which AI call produced which stub in test output:

```
// From src/ai/groq.js

export function makeProb(value, meta = {}) {

        return {

        __prob__:   true,

        value,

        confidence: meta.confidence ?? 1.0,

        model:   meta.model        ?? "unknown",

        task:      meta.task          ?? "",

        tokens:  meta.tokens        ?? 0,

        resolved:   false,

        };

}



export function stubAI(task, input, schema = null) {

        const value = schema

        ? Object.fromEntries(Object.keys(schema).map(k => [k, `[stub:${k}]`]))
```

```
        : `[AI stub: ${task}]`;

        return makeProb(value, { confidence: 1.0, model: "kova-stub", task });

}
```

makeProb and stubAI : creating Prob<T> values at runtime

6.7.4 resolveProb

The resolveProb() function is the runtime implementation of resolve(). It checks that the argument is a Prob<T> value using isProb(), marks it as resolved by setting v.resolved = true, and returns its inner value. If the argument is not a Prob<T>, it throws immediately with a clear error message:

```
// From src/ai/groq.js

export function resolveProb(v) {

        if (!isProb(v))

        throw new Error("resolve() expects a Prob<T> value, got: " + typeof v);

        v.resolved = true;

        return v.value;

}
```

resolveProb: the runtime implementation of resolve()

The resolved: true marking is metadata for tooling and debugging. In a future version of Kova, this field could be used to detect Prob<T> values that were created but never resolved, which would indicate a program that creates AI calls it never uses.

6.7.5 groqAI: The Live AI Call

When a Groq API key is present, groqAI() makes a real API call and wraps the result in a Prob<T>. It uses the Groq chat completions endpoint with the llama-3.3-70b-versatile model. The temperature is set to 0.2 to reduce variance in outputs, which is important for programs that branch on AI results.

```
// From src/ai/groq.js —----------> the live AI call

export async function groqAI(task, input, schema = null, apiKey = null) {

        const key = apiKey ?? process.env.GROQ_API_KEY;



        const systemPrompt = schema

        ? `You are a precise AI assistant. Respond ONLY with valid JSON

        matching this schema: ${JSON.stringify(schema)}.`

        : `You are a precise AI assistant. Respond with just the answer,

        no preamble.`;



        const body = {

        model:              "llama-3.3-70b-versatile",

        max_tokens:  512,

        temperature: 0.2,

        messages: [

        { role: "system", content: systemPrompt },

        { role: "user",   content: `Task: ${task}\nInput: ${input}` },

        ],
```

```
        };


        const res  = await fetch(GROQ_API_URL, { ... });

        const data = await res.json();

        const raw  = data.choices[0].message.content;


        return makeProb(raw.trim(), {

        confidence: 0.9,

        model:   data.model,

        task,

        tokens:  data.usage.total_tokens

        });

}
```

groqAI: the live Groq API call, condensed for clarity

The system prompt is crafted to minimise preamble in the AI's response. For a classification task, you want the model to return positive, not The sentiment of the provided text is positive. The prompt instructs the model to respond with just the answer. The temperature of 0.2 further reduces the likelihood of unexpected formatting.

6.7.6 Structured Output

When a schema is passed as the third argument to AI(), the system prompt instructs the model to respond with JSON matching that schema. The raw response is parsed with JSON.parse() after stripping any markdown code fences the model might add. If parsing fails, a clear error is thrown with the raw response included so the programmer can debug the prompt.

```
// Structured AI output in Kova

let probData = AI("extract fields", document, {

        name: "string",

        date: "string",

        amount: "string"

})



let data = resolve(probData)

// data is now: { name: "...", date: "...", amount: "..." }
```

Requesting structured JSON output from an AI call

6.7.7 Auto-Detection of Mode

The runKova() function in src/index.js automatically selects between live and stub mode based on whether GROQ_API_KEY is set in the environment. This means the same Kova program runs in stub mode during testing and development, and switches to live AI automatically when the key is present. No code changes required.

```
// From src/index.js

const aiMode = options.aiMode ?? (process.env.GROQ_API_KEY ? "groq" : "stub");



const aiExternals = {

        AI:       makeAIExternal(aiMode),   // live Groq or stub, based on mode

        resolve: resolveProb,
```

```
};
```

Auto-detection of AI mode based on environment variable

6.8 The Full Pipeline: End to End

Here is the complete journey of a single Kova program from source text to execution result:

```
// Source code

let review = "Outstanding quality!"

let prob   = AI("classify sentiment, respond lowercase", review)

let result = resolve(prob)

respond { status: 200, body: { sentiment: result } }
```

- **Lexer:** produces 20+ tokens including LET, IDENTIFIER(review), EQUALS, STRING, LET, IDENTIFIER(prob), EQUALS, IDENTIFIER(AI), LPAREN, STRING, COMMA, IDENTIFIER(review), RPAREN, and so on.
- **Parser:** produces a Program node with four children: two VariableDeclaration nodes, one with an AI CallExpression init, and one RespondStatement.
- **Semantic Analyzer:** declares review as string, declares prob as Prob<unknown>, checks that prob is not used directly in any expression, declares result as unknown after seeing resolve(prob), validates the respond object.
- **Interpreter:** evaluates review to the string value, calls AI() which returns a Prob<T> object, calls resolveProb() which returns the inner value, evaluates the respond object and stores it in this.respondValue.
- **Execution Graph:** produces a graph with declare nodes for review, prob, and result, an ai node for the AI() call, a resolve_prob node for resolve(), a respond node, and data edges tracking that prob flows into resolve_prob and result flows into respond.

- **Result:** returned to the caller as { returnValue: undefined, respondValue: { status: 200, body: { sentiment: "positive" } }, output: [], graph: { ... } }.

Key takeaway: *Every stage has a clear input and output. The lexer consumes characters and produces tokens. The parser consumes tokens and produces an AST. The semantic analyzer consumes the AST and produces validation. The interpreter consumes the AST and produces a result. The graph engine consumes the AST and produces a structural map. Each stage is independently testable and replaceable.*

Next: Section 7 - The CLI

The CLI

Kova programs are run through a command-line interface. This section covers installation, every available command, all output formats, error handling, and practical usage patterns.

7.1 Installation

7.1.1 Requirements

- Node.js version 18 or higher
- npm (included with Node.js)
- nvm recommended for managing Node versions

7.1.2 Installing from Source

```
# Clone the repository
git clone https://github.com/Cybugg/kova.git
cd kova

# Install dependencies
npm install
```

```
# Make the CLI file executable

chmod +x bin/kova.js



# Link globally so 'kova' works from any directory

npm link
```

Installing Kova from source

7.1.3 Verifying the Installation

```
kova help
```

```
Kova v0.5 : symbolic AI workflow language



Usage:

  kova run <file.kova>                  Run a Kova program

  kova run <file.kova> --graph   Run and display execution graph

  kova check <file.kova>               Parse and type-check without running

  kova help                        Show this message



Examples:
```

```
kova run examples/sentiment.kova

kova run examples/pipeline.kova --graph

kova check examples/api.kova
```

Expected output of kova help

7.1.4 nvm PATH Issue

If you use nvm and the kova command is not found after npm link, your shell's PATH may not include the nvm bin directory. Fix this permanently by adding the following to your ~/.bashrc or ~/.zshrc file:

```
echo 'export PATH="$(npm root -g)/../bin:$PATH"' >> ~/.bashrc

source ~/.bashrc
```

Permanent PATH fix for nvm users

After this, open a new terminal and run kova help to confirm the command is available.

7.2 Commands

Command	Description
kova run <file>	Run a Kova program with live AI
kova run <file> --graph	Run and display execution graph summary

kova check <file>	Parse and type-check without executing
kova help	Show the help message
kova --help	Same as kova help

7.3 kova run

7.3.1 Basic Usage

The run command reads a .kova file, runs it through the full pipeline, and prints all output to the terminal. It uses the async interpreter, which supports real Groq AI calls when GROQ_API_KEY is set.

```
kova run examples/sentiment.kova
```

```
[RESPOND] {"status":200,"body":{"approved":true,"sentiment":"positive"}}
```

```
[respond] {
  "status": 200,
  "body": {
        "approved": true,
        "sentiment": "positive"
  }
}
```

Output of kova run with Groq AI key present

The output has three possible components, printed in this order:

- **print() lines:** any output from print() calls in the program, printed first
- **[respond] block:** if the program called respond, the respond value is printed as formatted JSON
- **[return] value:** if the program called return at the top level, the return value is printed

7.3.2 Stub Mode

Without a GROQ_API_KEY set, Kova runs in stub mode. AI() calls return placeholder values instead of real AI responses. Everything else in the program runs normally:

```
[RESPOND] {"status":200,"body":{
 "sentiment":"[AI stub: classify sentiment as positive, negative, or neutral]"}}
```

Output in stub mode — placeholder values show the task string

Stub mode is the default for development and testing. You can write, run, and iterate on Kova programs without needing an API key or making any network calls.

7.3.3 File Path Resolution

File paths are resolved relative to the current working directory, not the location of the bin/kova.js file. Run Kova programs from your project root:

```
# From the kova project root
kova run examples/pipeline.kova
```

```
# From any directory with an absolute path

kova run /home/user/projects/myapp/pipeline.kova
```

File path resolution examples

7.4 kova run --graph

The --graph flag appends a full execution graph summary after the program output. This is the primary tool for understanding the structure of a Kova program:

```
kova run examples/pipeline.kova --graph
```

```
Environment/Science

Scientists have discovered a new method for capturing carbon...

Positive



[respond] {

 "status": 200,

 "body": {

        "category":  "Environment/Science",

        "summary":   "Scientists have discovered...",

        "sentiment": "Positive"

 }
```

```
}



#### Execution Graph ####

  Nodes          : 19

  Edges          : 30

  AI nodes   : 3

  Resolved   : 3

  Topological order:

        1. [entry] START

        2. [declare] let probCategory

        3. [declare] let probSummary

        4. [declare] let probSentiment

        5. [declare] let category

        ...

        13. [ai] AI(...)

        14. [ai] AI(...)

        15. [ai] AI(...)

        18. [respond] respond {...}

        19. [exit] END

  Parallel candidates: n13 || n14, n13 || n15, n14 || n15
```

Graph output for the pipeline example

7.4.1 Reading the Graph Output

The graph summary contains five sections:

- **Nodes:** total number of graph nodes. Each meaningful operation in the program is one node.
- **Edges:** total number of directed edges connecting nodes. Includes sequential, data, control, and binding edges.
- **AI nodes:** count of ai-kind nodes. Each AI() call is one. A well-formed program should have this equal to the number of AI() calls in the source.
- **Resolved:** count of resolve_prob-kind nodes. Should equal the number of AI nodes in a program that resolves all its AI calls.
- **Topological order:** the nodes listed in execution order. Each node's kind and label is shown. This is the order a sequential runtime would execute them.
- **Parallel candidates:** pairs of node IDs that could execute concurrently. The pipe symbol separates the two node IDs in each pair.

Key check: *If AI nodes and Resolved nodes are equal, every AI call in the program was resolved. If Resolved is less than AI nodes, some Prob<T> values were created but never resolved, a potential logic issue.*

7.5 kova check

The `check` command runs the lexer, parser, and semantic analyzer on a file without executing it. It is a fast way to validate a program's syntax and type correctness before running it:

```
kova check examples/api.kova
```

```
OK  examples/api.kova  (47 AST nodes)
```

Successful check output

If the program has a syntax or semantic error, check reports it with the source location:

```
KovaSemanticError: Equality operands must be the same type, got prob and string

--> Line 9, Column 14



 9 | if sentiment == "positive" {

                    ^
```

Error output from kova check

The check command is useful for:

- Validating a program before running it in production
- Catching type errors early in development without waiting for execution
- Verifying that Prob<T> values are all properly resolved before an AI call is made
- Counting AST nodes as a rough measure of program complexity

7.6 Setting Up the Groq API Key

7.6.1 Getting a Key

- Go to console.groq.com
- Sign up for a free account (no credit card required)
- Navigate to API Keys in the left sidebar
- Click Create API Key and give it a name
- Copy the key — it starts with gsk_

7.6.2 Adding the Key

Create a .env file in the root of your Kova project:

```
# .env

GROQ_API_KEY=gsk_your_key_here
```

The .env file

The CLI loads dotenv automatically. The key is read from the environment and used for all AI() calls in the program. You never need to reference it in your Kova code.

7.6.3 Keeping the Key Safe

- **.gitignore the .env file:** add .env to your .gitignore before committing
- **Never hardcode keys:** do not put the key directly in a .kova file or in source code
- **Use ENV variables for all secrets:** the ENV.KEY pattern works for any secret, not just the Groq key

```
# .gitignore

.env

*.env
```

Add .env to .gitignore

7.7 Error Messages

Kova produces three categories of error messages. Each has a distinct format:

7.7.1 Syntax Errors

Produced by the parser when the token stream does not match any valid grammar rule:

```
KovaSyntaxError: Expected RBRACE but got EOF at line 7, col 1
```

Syntax error — usually a missing closing brace

7.7.2 Semantic Errors

Produced by the semantic analyzer. Always include the source line and a caret pointing to the exact position:

```
KovaSemanticError: Undeclared variable "userName"

--> Line 3, Column 20



  3 | let greeting = userName + " welcome"

                     ^
```

Semantic error with source location

8.7.3 Runtime Errors

Produced during execution. Include the error message and exit with code 1:

```
RuntimeError: Division by zero
```

The CLI catches all three error types, prints them to stderr, and exits with code 1. This makes Kova programs easy to integrate into shell scripts and CI pipelines that check exit codes.

7.8 Using Kova Programmatically

In addition to the CLI, Kova can be used as a JavaScript module. The src/index.js file exports three functions:

```
import { runKova, runKovaSync, parseKova } from "./src/index.js";


// Async: uses Groq if key is present, stub otherwise
const result = await runKova(`
        let p = AI("classify", "great product")
        let v = resolve(p)
        respond { status: 200, body: { label: v } }
`);


console.log(result.respondValue);   // { status: 200, body: { label: "positive" } }
console.log(result.graph.json.nodes.length);   // number of graph nodes


// Sync:stub AI only, no network calls
const syncResult = runKovaSync(`let x = 5  return x`);
console.log(syncResult.returnValue);   // 5
```

```
// Parse only — returns AST and graph without executing
const { ast, graph } = parseKova(`let x = 5 + 3`);
```

Using Kova as a JavaScript module

The programmatic API is what the test suite uses. Every test calls runKovaSync() with a Kova code string and checks the result. This makes it straightforward to embed Kova in larger systems, use it as a scripting layer, or build tooling on top of it.

7.9 Running the Test Suite

```
npm test
```

```
Variable Declaration
  v number
  v string
  v boolean
  ...

AI Integration (Prob<T>) -- Sync
  v AI() returns a Prob value
```

```
 v resolve() unwraps Prob to value

 ...



############################################################

 117 passed, 0 failed out of 117 tests (100.0%)

############################################################
```

npm test output with all tests passing

The test suite uses runKovaSync() throughout, which means it does not require a Groq API key and runs entirely offline. All 117 tests complete in under a second.

7.10 Project Structure Reference

```
kova/
├── bin/
│   └── kova.js            CLI entry point
├── src/
│   ├── index.js           Public API — runKova, runKovaSync, parseKova
│   ├── lexer/lexer.js     Tokenizer
│   ├── parser/parser.js  Recursive-descent parser
│   ├── semantic/
│   │   └── semanticAnalyzer.js  Type checker + Prob<T> enforcement
│   ├── interpreter/
```

```
│   │   └── interpreter.js   Async tree-walk interpreter
│   ├── graph/
│   │   └── executionGraph.js    Execution graph engine
│   ├── ai/
│   │   └── groq.js        Groq client + Prob<T> runtime
│   └── core/
│           └── diagnostic.js          Error formatting
├── lib/
│   ├── constants/store.js   Keywords, operators, symbols
│   ├── functions/index.js   Built-in function signatures
│   └── regex/index.js   Character class regexes
├── examples/
│   ├── sentiment.kova
│   ├── pipeline.kova
│   ├── content-moderation.kova
│   ├── support-router.kova
│   ├── document-enrichment.kova
│   ├── user-onboarding.kova
│   ├── contract-analysis.kova
│   └── health-check-service.kova
├── tests/
│   └── kova.test.js       117-test suite
├── .env                   API keys (not committed)
```

```
├── package.json
└── README.md
```

Complete Kova project structure

Next: Section 8 - Limitations

Limitations

Every tool has limits. Kova is a research-oriented proof of concept, not a production platform. This section documents every significant limitation honestly, distinguishes between limitations that are fundamental to Kova's design and those that could be addressed in future versions, and explains why each limitation exists.

Three categories of limitation are used throughout this section:

- **Design limitations:** deliberate constraints that follow from Kova's purpose and scope. These are not bugs.
- **Current implementation limitations:** things Kova does not yet do but could, given more development time.
- **Hard limitations:** things Kova cannot do and is not intended to do.

8.1 Language Scope

Kova is not a general-purpose language

> Kova is designed for one domain: AI-integrated backend workflows. It is not suitable for systems programming, desktop applications, data science, game development, or any domain outside its intended scope. This is deliberate. Narrowness is what makes Kova's type system meaningful.

There are things you cannot write in Kova that you can write in Python or JavaScript:

- File system operations like reading or writing files on disk
- Multi-threading or concurrent execution within a program
- Low-level memory management
- GUI applications or frontend web interfaces

- Native compiled executables
- Networking beyond HTTP — WebSockets, TCP, UDP
- Image, audio, or binary data processing

None of these are Kova's territory. The language is narrow by design, and the correct response to needing these capabilities is to use a general-purpose language.

8.2 Runtime and Performance

Kova uses a tree-walk interpreter

> Tree-walk interpreters are slow compared to compiled languages or even bytecode-compiling interpreters. Every loop iteration re-traverses the AST. For computationally intensive programs, this is a significant bottleneck.

Concretely, Kova is not suitable for:

- High-throughput production services handling thousands of requests per second
- CPU-intensive computation like large loops, numerical algorithms, data transformation at scale
- Real-time systems where response latency must be measured in microseconds

For the use cases Kova targets backend workflows that spend most of their time waiting for AI API responses and database queries, so interpreter overhead is less significant. The bottleneck in those programs is network latency, not execution speed.

Future direction: *A bytecode compiler for Kova would significantly improve performance. The AST structure is well-suited to bytecode compilation, and this is identified as a future direction in Section 9.*

8.3 The Module System

Imports are parsed but not fully executed

> Kova's import syntax is parsed and recorded in the AST. Imported names are registered so the semantic analyzer does not reject programs that reference them. But the actual module loading, reading the imported file, executing it, and binding its exports, is not implemented in the current interpreter.

This means:

- You can write import statements and the program will not throw
- Imported names are treated as external references that pass type checking
- The actual values from imported modules are not available at runtime

For the example programs and test suite, this is not an issue because they do not rely on module exports. For building larger programs that split logic across files, it is a meaningful gap.

8.4 HTTP and Database Stubs

HTTP requests and database operations are stubbed

> Kova's HTTP verbs and database statements produce realistic-looking result objects, but they do not make real network requests or connect to real databases. They are stubs that allow programs to be written and tested with the correct structure.

Specifically:

- GET, POST, PUT, DELETE, PATCH: log the call and return a stub response object with ok: true and status: 200. No real HTTP request is made.
- connect: records the connection configuration and returns a stub connection object. No real database driver is loaded.
- find, insert, update: return stub result objects with plausible field values. No real query is executed.

The architecture is designed for real implementations to be injected. The interpreter accepts an externals object at construction time, and HTTP and database handlers could be provided as externals that make real calls. Building these handlers is identified as a future direction.

> **Note:** *AI() is the one domain-specific operation that IS fully implemented with a real backend. The Groq integration is complete and production-quality.*

8.5 Type System Completeness

Kova's type system is hybrid and partial

> Kova is dynamically typed for general values and statically typed only for Prob<T>. This is intentional: the goal is to enforce AI uncertainty handling, not to build a complete static type system. But it means many type errors that a fully static language would catch at compile time are only caught at runtime in Kova.

Examples of type errors Kova does not catch statically:

```
let x = 5

let y = x + "hello"   // runtime error: cannot add number and string

                      // caught at runtime, not by the semantic analyzer
```

A type error Kova catches only at runtime

The semantic analyzer catches undeclared variables, incorrect argument counts, and all Prob<T> violations. It does not perform full type inference or check arithmetic type compatibility for non-Prob values.

A complete static type system for Kova would be a significant undertaking and is beyond the scope of the current research prototype.

8.6 Error Handling

No try-catch or structured error handling

> Kova has no try-catch syntax. Runtime errors terminate the program immediately. There is no way to recover from an error within a Kova program.

This means:

- A division by zero stops the entire program
- A failed resolve() on a non-Prob value stops the program
- An HTTP stub that encounters an unexpected input stops the program

For a research language demonstrating type system concepts, this is acceptable. For a production service, it would not be. Error handling syntax is straightforward to add to the parser and interpreter and is identified as a near-term addition.

8.7 The Parallel Execution Gap

Parallel candidates are identified but not executed in parallel

> The execution graph identifies pairs of operations that could run concurrently with no correctness risk. But the interpreter executes everything sequentially. The parallel candidate information is available for analysis but not acted upon by the runtime.

This is the most significant gap between what Kova identifies and what it currently does. Three independent AI() calls in a program are identified as parallel candidates, but the interpreter awaits each one in sequence, making three separate API calls one after another rather than simultaneously.

```
// These three calls are sequential in the current runtime

// even though the graph identifies them as parallel candidates
```

```
let probA = AI("task A", input)

let probB = AI("task B", input)

let probC = AI("task C", input)



// A concurrent runtime would send all three to Groq simultaneously

// and wait for all three to complete, rather than waiting for each in turn
```

Parallel AI calls that the current runtime executes sequentially

Implementing parallel execution for independent AI calls would likely reduce execution time by a factor of two to three for programs with multiple AI calls. The execution graph already contains the information needed to do this correctly. The runtime just needs to act on it.

8.8 No Package Ecosystem

Kova has no package manager and no third-party library ecosystem

> There is no equivalent of npm or pip for Kova. The language ships with what it ships with. You cannot install external libraries, use community-built utilities, or publish packages for others to use.

Everything available in a Kova program is in this manual. The built-in functions, the HTTP and database stubs, the AI integration. There is nothing beyond what is documented here.

Building a package ecosystem for a research language at this stage would be premature. The language needs to prove its core ideas first.

8.9 Single AI Provider

Only Groq is supported as an AI backend

> The current implementation connects to Groq's API using the llama-3.3-70b-versatile model. There is no support for OpenAI, Anthropic, Google, or any other provider. There is no way to configure which model is used without modifying the source code.

The AI layer is architected to be replaceable. The groqAI function in src/ai/groq.js is the only provider-specific code. A provider-agnostic interface with a configuration option for model selection is straightforward to build on top of the current architecture.

8.10 The Prob<T> Confidence Field

Confidence values are not meaningfully used

> Every Prob<T> object has a confidence field. The Groq integration sets this to 0.9 for all responses. The stub sets it to 1.0. Neither value is meaningful. The field exists in the object and can be read by a program, but it does not reflect a real probability estimate.

A meaningful confidence value would require either a model that reports calibrated uncertainty estimates (most current LLMs do not), or a post-processing step that estimates confidence from the model's output distribution. This is a research problem in its own right.

The field is present as a placeholder for when the problem is solved. Programs that read it today will continue to work when meaningful values are introduced.

8.11 What Kova Is Not Trying to Fix

Some limitations are not limitations of Kova but of the underlying problem. It is important to be clear about these:

- **AI model accuracy:** Kova cannot make AI models more accurate. A model that classifies sentiment incorrectly will still classify incorrectly in Kova. Kova ensures the program handles this correctly. It cannot prevent it.

- **AI bias and fairness:** Kova does not address bias in AI model outputs. A biased model will produce biased outputs. Kova tracks that the output came from AI. It does not evaluate whether the output is fair.
- **Prompt injection:** a malicious input could attempt to manipulate the AI model into ignoring its task instructions. Kova does not sanitise AI inputs or defend against prompt injection attacks.
- **Model hallucination:** AI models can produce confident but factually incorrect outputs. Kova's Prob<T> type acknowledges uncertainty but does not detect hallucinations.

These are real problems. They are problems that operate at the model level, the data level, or the deployment level, not the programming language level. Kova addresses the programming language level.

Summary: *Kova's limitations are real, documented, and mostly addressable. The core idea that AI uncertainty should be a structural property of programs is not limited. The current implementation is a research prototype that demonstrates the idea. Future versions can remove the implementation limitations one by one.*

Next: Section 9 - What Kova Could Be

What Kova Could Be

Kova v0.5 demonstrates a core idea: AI uncertainty can be a first-class structural property of a programming language. That demonstration is complete. What follows is a map of where Kova could go from here.

This section is divided into three time horizons. Near-term directions are extensions to the current implementation that require engineering work but no new research. Medium-term directions require more significant architectural changes. Research directions are open questions that connect Kova to active areas of academic research.

Three colours are used to mark each direction:

- **Green (Near-term):** engineering work, addressable in weeks to months
- **Blue (Medium-term):** architectural work, addressable in months to a year
- **Purple (Research direction):** open research questions, timeline measured in research cycles

9.1 Near-Term: Completing the Implementation

Near-term: Parallel AI execution

The execution graph already identifies which AI() calls have no dependencies on each other. The runtime should act on this information by sending independent AI requests concurrently using Promise.all(). This is the highest-value near-term addition. Programs with three independent AI calls would run roughly three times faster.

The change is localised to the interpreter. When visiting a block of statements, the interpreter would scan ahead, identify groups of independent AI calls, and execute them concurrently before proceeding. The execution graph's parallel candidates already contain exactly the information needed.

```
// Current behaviour: three sequential API calls

let pA = AI("classify category",  article)   // wait...

let pB = AI("summarize",              article)   // wait...

let pC = AI("classify sentiment",  article)   // wait...



// Future behaviour: one concurrent round

// All three sent simultaneously, results collected together

// Total time = max(tA, tB, tC) instead of tA + tB + tC
```

The performance impact of parallel AI execution

Near-term: **Real HTTP execution**

Replace the HTTP stub with an actual fetch() implementation. The interpreter already has the request method, URL, and body. Wiring these to Node.js's built-in fetch is straightforward. This would make Kova programs genuinely useful for integrating real APIs.

Near-term: **Real database drivers**

Implement database adapters for at least one database, postgres recommended given its prevalence. The connect, find, insert, and update statements already produce the correct AST nodes and pass through the interpreter. The missing piece is replacing the stub result objects with real query execution.

Near-term: **try-catch error handling**

Add try and catch keywords to the parser and interpreter. The error handling model can be simple: try { } catch err { } where err is bound to the error message. This removes the current limitation where any runtime error terminates the program.

```
// Proposed try-catch syntax
try {
    GET "https://api.example.com/data" into result
    let prob = AI("classify", result)
    let label = resolve(prob)
    respond { status: 200, body: { label: label } }
} catch err {
    respond { status: 500, body: { error: err } }
}
```

Proposed try-catch syntax for Kova

Near-term: Multi-provider AI support

Add a provider configuration to the AI layer so programs can specify which model to use. Support at minimum OpenAI and Anthropic alongside Groq. The groqAI function is already isolated from the rest of the interpreter, making provider switching a contained change.

Near-term: Module system implementation

Implement the import statement fully. When the interpreter encounters an import, it reads the specified file, runs it through the pipeline, and binds the exported names into the current scope. This enables splitting large Kova programs across multiple files.

9.2 Medium-Term: Architectural Extensions

Medium-term: **Bytecode compiler**

Replace the tree-walk interpreter with a bytecode compiler and virtual machine. The AST is compiled to a sequence of bytecode instructions. The VM executes the bytecode. This would significantly improve performance for compute-intensive programs and enable optimisations like constant folding and dead code elimination.

Medium-term: **Effect system for AI calls**

Extend the type system to track not just whether a value is Prob<T> but what kind of AI effect it carries: which model was used, what task was performed, what confidence level is associated. This would enable more precise static reasoning about AI interactions and support more detailed audit trail generation.

```
// Proposed effect annotation syntax

let prob: Prob<string, groq, classify> = AI("classify", text)

//                        ^model  ^task



// The type system could then enforce:

// - only resolve Prob values from trusted models

// - require confidence above a threshold before using in critical branches
```

Hypothetical effect-annotated Prob<T> type

Medium-term: **Execution graph visualiser**

Build an interactive HTML visualisation of the execution graph using D3.js. The graph JSON is already produced by every program run. A web-based viewer that renders it as an interactive diagram would make the structural properties of Kova programs immediately visible and navigable.

Medium-term: **Static analysis tooling**

Build a linter and static analysis tool that operates on the AST and execution graph without running the program. It could detect: AI calls with no corresponding resolve(), resolve() calls that are never used in a condition or response, parallel AI call opportunities that the programmer missed, and overly broad task strings likely to produce inconsistent outputs.

Medium-term: **Language server protocol support**

Implement the Language Server Protocol so that editors like VS Code can provide Kova-aware autocomplete, hover documentation, and inline error highlighting. The semantic analyzer already produces the type information needed. Wrapping it in an LSP server would bring Kova into the modern editor experience.

9.3 Research Directions

Research direction: **Execution graphs as POMDP policies**

Formalise the connection between Kova execution graphs and Partially Observable Markov Decision Processes. A Kova program that calls AI() to estimate state and acts on that estimate is structurally a POMDP policy. If this connection can be made precise, techniques from reinforcement learning research on POMDPs become applicable to the analysis and optimisation of Kova programs.

Concretely: given a Kova program and a distribution over possible AI outputs, could a Kova execution graph be used to compute the expected outcome of the program? Could it be used to

identify which AI calls most influence the program's behaviour and should therefore be made most carefully?

Research direction: **Symbolic scaffolding for AI improvement**

Explore whether the symbolic structure of a Kova execution graph can be used to improve the AI calls within it. If the graph encodes the constraints that the AI output must satisfy to produce a correct program execution, those constraints could be used as a reward signal to fine-tune the AI model toward outputs that satisfy them.

This is the deepest research direction Kova points toward: using the symbolic program as a specification that guides the improvement of the neural components it calls. The execution graph is the interface between the two.

Research direction: **Calibrated confidence in Prob<T>**

The confidence field in Prob<T> currently holds a placeholder value. Making it meaningful requires either models that report calibrated uncertainty or post-hoc calibration methods. Research on language model uncertainty quantification, including conformal prediction and temperature scaling, could be applied here to give Prob<T> a genuine probabilistic interpretation.

Research direction: **Formal verification of AI-integrated programs**

Develop a formal semantics for Kova that supports reasoning about programs containing probabilistic components. Given a Kova program, a formal model of the AI's output distribution, and a specification of correct behaviour, could a theorem prover verify that the program satisfies the specification with high probability? The execution graph is the natural representation for such a proof.

Research direction: **Influence on mainstream languages**

> The most impactful direction for Kova is not Kova itself but what Kova demonstrates to the designers of mainstream languages. Python, JavaScript, and TypeScript all have growing AI integrations. None of them have a structural mechanism for handling AI uncertainty. Kova is a proof that such a mechanism is possible and practical. Publishing the design and findings as a research paper is the path to influencing language design at scale.

9.4 The Longer Vision

The goal is not a better language. The goal is a better way of thinking about programs that use AI.

Kova started as a question: can a programming language's type system make AI uncertainty structurally visible? The answer is yes. Kova proves it.

But the deeper question is whether this approach can scale. Can the ideas in Kova influence how Python handles AI library calls? Can they influence how TypeScript's type system evolves as AI becomes more central to JavaScript development? Can they find their way into language standards proposals?

That is the longer vision. Kova as a proof of concept that earns its ideas a place in the mainstream discussion. A language that asks a question clearly enough that the question cannot be ignored.

The execution graph points toward something even further. If a Kova program is a policy and its execution graph is a symbolic representation of that policy, then the graph is a bridge between the world of formal programming languages and the world of learning systems. That bridge has not been built. Building it is the research agenda Kova opens.

> **In summary:** *Kova v0.5 is a beginning. The near-term directions complete the implementation. The medium-term directions make it production-relevant. The research directions make it scientifically significant. All three are worth pursuing.*

Next: Section 10 - Appendix and Glossary

Appendix and Glossary

Below is the reference section: full keyword list, table of operators, error reference, quick start guide, full glossary, and bibliography. Use this as a resource when writing Kova programs or when trying to understand the source code.

A.1 Complete Keyword Reference

Keyword	Token type	Description
let	LET	Variable declaration
fn	FN	Function declaration
return	RETURN	Return from function or program
if	IF	Conditional branch
else	ELSE	Alternate conditional branch
while	WHILE	Loop while condition is true
for	FOR	Iterate over array
in	IN	Part of for...in syntax
import	IMPORT	Import from another module
from	FROM	Part of import syntax

export	EXPORT	Export a value (parsed, not executed)
GET	GET_HTTP	HTTP GET request
POST	POST_HTTP	HTTP POST request
PUT	PUT_HTTP	HTTP PUT request
DELETE	DELETE_HTTP	HTTP DELETE request
PATCH	PATCH_HTTP	HTTP PATCH request
into	INTO	Bind result to a variable
save	SAVE	Attach body to HTTP or DB operation
connect	CONNECT	Open a database connection
using	USING	Provide configuration to connect
find	FIND	Query records from a collection
insert	INSERT	Insert a record into a collection
update	UPDATE	Update records in a collection
where	WHERE	Filter condition for DB queries
set	SET	Fields to update in an update statement
limit	LIMIT	Maximum number of records to return
order_by	ORDER_BY	Sort field for find queries
asc	ASC	Ascending sort direction

desc	DESC	Descending sort direction
respond	RESPOND	Send response and end execution
ENV	ENV	Access environment variables via ENV.KEY
true	BOOLEAN	Boolean literal true
false	BOOLEAN	Boolean literal false
null	NULL	Null literal

A.2 Complete Operator Reference

Op	Name	Notes
+	Addition / concat	Adds numbers or concatenates strings
-	Subtraction	Numbers only
*	Multiplication	Numbers only
/	Division	Throws on divide by zero
%	Modulo	Remainder after division
-x	Unary negation	Negates a number
==	Equality	Strict equality (===)

!=	Inequality	Strict inequality (!==)
>	Greater than	
<	Less than	
>=	Greater or equal	
<=	Less or equal	
&&	Logical AND	Both operands must be boolean
||	Logical OR	Both operands must be boolean
!	Logical NOT	Inverts a boolean
=	Assignment	Assign to existing variable
+=	Add-assign	x += n is x = x + n
-=	Subtract-assign	x -= n is x = x - n
*=	Multiply-assign	x *= n is x = x * n
/=	Divide-assign	x /= n is x = x / n

A.3 Error Message Reference

Semantic Errors

Undeclared variable "name"

A variable was referenced before being declared with let. Check spelling and scope. Variables declared inside a block are not visible outside it.

Variable "name" already declared in this scope

A variable with this name was already declared in the same block. Use a different name or remove the duplicate declaration.

Equality operands must be the same type, got prob and string

A Prob<T> value was used directly in a comparison. Call resolve() on the Prob<T> value first, then compare the result.

Probabilistic values must be resolved before use

A Prob<T> value was used in an arithmetic expression or other operation that requires a concrete value. Call resolve() first.

Probabilistic value cannot be used in 'if' condition. Use resolve().

A Prob<T> value was used directly as an if condition. Call resolve() and use the result in the condition.

resolve() expects a probabilistic value

resolve() was called with a non-Prob value. Only values returned by AI() can be passed to resolve().

Invalid argument count for "fn": expected N, got M

A built-in or user-defined function was called with the wrong number of arguments.

Type mismatch: declared T but got U

A type annotation was specified on a variable declaration and the initialiser returned a different type.

Runtime Errors

Division by zero

A division or modulo operation was performed with a denominator of zero.

Undefined variable "name"

A variable that passed semantic analysis was not found at runtime. This typically means a variable was declared in a branch that was not taken.

resolve() expects a Prob<T> value, got: typename

The runtime version of the resolve() error. The value passed to resolve() at runtime was not a Prob<T> object.

len() expects array or string

The len() built-in was called with a value that is neither an array nor a string.

A.4 Quick-Start Cheat Sheet

```
// ── Setup ──────────────────────────────────────────────────────────────────

// Install: git clone, npm install, chmod +x bin/kova.js, npm link

// Run:   kova run <file.kova>

// Check:   kova check <file.kova>

// Graph:   kova run <file.kova> --graph

// AI key:  echo 'GROQ_API_KEY=gsk_...' > .env



// ── Variables ──────────────────────────────────────────────────────────────

let name  = "Alice"

let count = 42

let flag  = true

count += 1



// ── Control flow ───────────────────────────────────────────────────────────

if count > 10 { print("big") } else { print("small") }

while count > 0 { count -= 1 }

for item in [1, 2, 3] { print(item) }

for i in range(0, 5) { print(i) }



// ── Functions ──────────────────────────────────────────────────────────────
```

```
fn greet(name) { return "Hello " + name }

print(greet("Alice"))



// ── AI (the core feature) ──────────────────────────────────────────

let prob   = AI("classify sentiment as positive or negative", text)

let result = resolve(prob)              // must resolve before use

if result == "positive" { respond { status: 200, body: { ok: true } } }



// ── HTTP ──────────────────────────────────────────────────────────────

GET  "https://api.example.com/data" into res

POST "/users" save { name: "Alice" } into created



// ── Database ──────────────────────────────────────────────────────────

connect postgres using { host: ENV.DB_HOST, database: "app" }

find   users where { active: true } limit 10 into results

insert into users { name: "Alice" } into newUser

update users set { verified: true } where { id: userId }



// ── Respond ───────────────────────────────────────────────────────────

respond { status: 200, body: { data: results } }
```

Kova quick-start cheat sheet; all major features on one page

A.5 Glossary

All terms are defined as they are used in Kova's context.

Abstract Syntax Tree (AST)

A tree data structure that represents the structure of a program. Each node in the tree corresponds to a construct in the source code. The AST is produced by the parser from a list of tokens and consumed by the semantic analyzer and interpreter.

AI node

A node kind in the Kova execution graph. Every AI() call in a program produces one ai node. The count of ai nodes tells you how many AI calls were made in a program run.

Bytecode

An intermediate representation of a program, between source code and machine code. Bytecode is designed to be efficient to execute by a virtual machine. Kova currently uses a tree-walk interpreter rather than bytecode, but a bytecode compiler is identified as a future direction.

Calibrated uncertainty

A confidence score is calibrated if it accurately reflects the probability of being correct. A model that says 90% confident should be right approximately 90% of the time. Current language models generally do not produce well-calibrated confidence scores.

Closure

A function that captures variables from its enclosing scope. In Kova, functions can access variables declared in the outer scope where the function was defined, not just variables in the function's own scope.

Compiler

A program that translates source code into another form, typically machine code or bytecode, before execution. Kova uses an interpreter, not a compiler, though a bytecode compiler is a future direction.

Control flow graph (CFG)

A directed graph representation of a program where nodes are basic blocks of code and edges represent possible control transfers. Kova's execution graph extends the CFG concept with domain-specific node kinds including ai and resolve_prob.

Data dependency edge

An edge in the execution graph with kind 'data'. It connects the node that produces a value to the node that consumes it. The edge label is the variable name. Data dependency edges enable the graph engine to identify which operations are truly independent and could run in parallel.

Domain-specific language (DSL)

A programming language designed for a specific problem domain rather than general computation. SQL is a DSL for database queries. Kova is a DSL for AI-integrated backend workflows.

Dynamic typing

A type system where types are checked at runtime rather than before execution. Kova is mostly dynamically typed, with the exception of Prob<T> which is checked statically by the semantic analyzer.

Effect system

An extension of a type system that tracks not just what type a value has but what computational effects its computation may have. Kova's Prob<T> can be understood as a lightweight effect annotation, and a full effect system for AI calls is identified as a future direction.

Execution graph

A directed graph produced by every Kova program run. Nodes represent operations. Edges represent sequential, data, control, call, or binding relationships. The graph makes the structure of a program, including where AI uncertainty enters and exits, visible and inspectable.

Externals

Functions injected into the Kova interpreter at construction time. AI(), resolve(), and all built-in functions that require external resources are provided as externals. This allows the interpreter to be used with real or stub implementations of these functions.

Information flow typing

A type system that tracks where data comes from and where it is allowed to go. Used primarily for security to prevent secret data from flowing to public outputs. Kova applies a structurally similar idea to track AI-produced uncertainty through a program.

Interpreter

A program that reads source code and executes it directly, without first compiling to machine code. Kova uses a tree-walk interpreter that recursively visits AST nodes and executes the operation each node represents.

Lexer

The first stage of the Kova pipeline. The lexer reads source code character by character and produces a flat list of tokens. Also called a tokenizer or scanner.

Markov Decision Process (MDP)

A mathematical framework for modelling sequential decision making under uncertainty. A Kova program that calls AI() to estimate state and acts on that estimate is structurally analogous to an MDP policy.

Neuro-symbolic computing

A research area combining neural networks with symbolic reasoning systems. Kova's execution graph, which integrates symbolic program structure with neural AI calls, is a contribution to this area.

npm link

A Node.js command that creates a global symlink from the kova command to the bin/kova.js file in the project directory. Required to use the kova CLI from any directory.

Parallel candidates

Pairs of nodes in the execution graph that have no dependency relationship, either direct or transitive, and could therefore be executed simultaneously without affecting the result. Currently identified but not acted upon by the runtime.

Parser

The second stage of the Kova pipeline. The parser takes the token list from the lexer and builds an Abstract Syntax Tree. Kova uses a recursive-descent parser where each grammar rule corresponds to one parsing function.

Partially Observable MDP (POMDP)

An extension of the MDP framework where the agent cannot directly observe the true state of the world. It must maintain a probability distribution over possible states. A Kova program that uses AI() to make observations in an uncertain world is structurally a POMDP policy.

Probabilistic programming language (PPL)

A programming language designed to express probabilistic models, where variables represent probability distributions and inference is a first-class operation. Church, Anglican, Stan, and Pyro are examples. Kova is related but distinct: it enforces uncertainty acknowledgment rather than enabling probabilistic inference.

Prob<T>

The probabilistic type in Kova. Every AI() call returns a value of type Prob<T>. It is a plain JavaScript object with fields __prob__, value, confidence, model, task, tokens, and resolved. It cannot be used in expressions or

conditions without first being unwrapped by resolve(). The <T> represents the inner type of the value, which is unknown until resolution.

Recursive-descent parser

A top-down parsing technique where each grammar rule in the language is implemented as a function that calls other rule functions recursively. Kova's parser is a recursive-descent parser. It is the most readable and understandable parsing technique for hand-written parsers.

resolve()

The built-in function that unwraps a Prob<T> value. It takes a Prob<T> and returns the concrete value inside it. It marks the Prob<T> as resolved. Calling resolve() on a non-Prob value throws a runtime error.

resolve_prob node

A node kind in the Kova execution graph. Every resolve() call produces one resolve_prob node. The count of resolve_prob nodes should equal the count of ai nodes in a well-formed program where all AI calls are resolved.

Runtime error

An error that occurs during program execution rather than during semantic analysis. Division by zero and calling resolve() on a non-Prob value are runtime errors in Kova. Runtime errors terminate the program immediately.

Scope

The region of a program where a variable is visible. In Kova, variables are scoped to the block they are declared in. The scope stack is a data structure used by both the semantic analyzer and interpreter to track which variables are visible at any point during analysis or execution.

Semantic analysis

The third stage of the Kova pipeline. The semantic analyzer checks that the program makes sense: variables are declared before use, functions are called with the right arguments, and Prob<T> values are never used without resolution. Semantic errors are caught before execution.

Semantic error

An error detected by the semantic analyzer before the program runs. Semantic errors are reported with the source line and column number of the offending construct.

Stub mode

The operating mode Kova uses when no GROQ_API_KEY is set. AI() calls return placeholder values instead of making real API requests. All other language features work normally. Stub mode is the default for development and testing.

Symbol table

A data structure that maps variable names to their types (in the semantic analyzer) or values (in the interpreter). The scope stack is a stack of symbol tables.

Token

The smallest meaningful unit of a Kova program. Produced by the lexer. Examples include keywords (let, if, AI), identifiers (userName, count), operators (==, +=), literals (42, "hello"), and punctuation ({, }, ,).

Topological order

A linear ordering of graph nodes such that every node appears after all the nodes it depends on. The execution graph engine computes a topological order for every Kova program, giving a valid sequential execution order.

Tree-walk interpreter

An interpreter that executes a program by recursively walking its Abstract Syntax Tree, visiting each node and performing the operation it represents. Kova uses a tree-walk interpreter. It is simple and extensible but slower than bytecode-compiling interpreters.

Type system

A set of rules that assigns types to values and expressions in a program, and checks that operations are applied to appropriate types. Kova has a hybrid type system: dynamically typed for general values and statically checked for Prob<T>.

A.6 Bibliography

The following works are cited in this manual or are directly relevant to Kova's design and research context.

Abadi, M. et al. (2016). *TensorFlow: A system for large-scale machine learning.* 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), pp. 265-283.

Abelson, H. and Sussman, G.J. (1996). *Structure and Interpretation of Computer Programs (2nd ed.).* MIT Press.

Ball, T. (2018). *Writing an Interpreter in Go.* Independently published.

Casado, R. and Younas, M. (2015). *Emerging trends and technologies in big data processing.* Concurrency and Computation: Practice and Experience, 27(8), pp. 2078-2091.

Denning, D.E. and Denning, P.J. (1977). *Certification of programs for secure information flow.* Communications of the ACM, 20(7), pp. 504-513.

Doshi-Velez, F. and Kim, B. (2017). *Towards a rigorous science of interpretable machine learning.* arXiv:1702.08608.

Goodman, N.D. et al. (2012). *Church: A language for generative models.* Proceedings of the 24th Conference on Uncertainty in Artificial Intelligence (UAI).

Kaelbling, L.P., Littman, M.L. and Cassandra, A.R. (1998). *Planning and acting in partially observable stochastic domains.* Artificial Intelligence, 101(1-2), pp. 99-134.

Karpathy, A. (2015). *The Unreasonable Effectiveness of Recurrent Neural Networks.* Blog post. karpathy.github.io/2015/05/21/rnn-effectiveness.

Mao, J., Gan, C., Kohli, P., Tenenbaum, J.B. and Wu, J. (2019). *The neuro-symbolic concept learner: Interpreting scenes, words, and sentences from natural supervision.* International Conference on Learning Representations (ICLR).

Marcus, G. (2019). *The next decade in AI: Four steps towards robust artificial intelligence.* arXiv:2002.06177.

Nystrom, R. (2021). *Crafting Interpreters.* Genever Benning. craftinginterpreters.com.

Pierce, B.C. (2002). *Types and Programming Languages.* MIT Press.

Pfeffer, A. (2016). *Practical Probabilistic Programming.* Manning Publications.

Tran, D. et al. (2017). *Deep probabilistic programming.* International Conference on Learning Representations (ICLR).

End of Manual

Kova v0.5.0 | Samuel Ifedapo Oguntimehin | March 2026

github.com/kova-lang